

# BTRON Concurrency Tiers. MP/AMP/SMP/Async, Media RTP and the InterCore in the Calculus of Inductive Constructions, with Modal Extensions for Non-Determined Multicore Parallelism

Maxim Sokhatsky (Namdak Tonpa)

*Synrc Research Center / Groupoid Infinity / BTRON Cleanroom Working Group*  
*Kyiv, Ukraine*  
maxim@synrc.com

September 2026

## Abstract

The cleanroom reconstruction of B-System (BTRON 3.20) has produced, alongside the C kernel, a second artefact that is easy to overlook: a body of mechanical mathematics in which the concurrency, media, and inter-core subsystems are not described but *defined*, and in which their properties are not argued for but *computed*. This paper presents that body of work as a single mathematical model. We take as our object language the Calculus of Inductive Constructions, and as our single rule of inference the judgemental equality  $t = u$  discharged by **reflexivity**. Four claims are made. First, that the apparently heterogeneous tiers of the BTRON scheduling model—multi-processing, asymmetric multiprocessing, symmetric multiprocessing, and the asynchronous I/O plane—are four instances of one algebraic signature, a total state transformer  $S \rightarrow S \times R$  over a table-shaped bus, differing only in which coordinates of the state their guards read. Second, that the media stack, from NuStreams abstraction layers through RTP packetisation and loss accounting to the symmetric multi-plane pipeline, is equationally a *fold over a watermark*: every streaming invariant we state, including the sequence-number, jitter, and residence-time laws, reduces to arithmetic on counters that are themselves derived as lengths and differences of lists, never stored. Third, that the InterCore protocol—**pub**, **sub**, **spawn**, **snd**, **rcv** over a star of single-producer queues—admits a broker-free construction in which the multi-consumer view is a pure function of the producer views, so that the entire sharing discipline collapses to one equation about cursor ownership and its corollary, the absence of mutual exclusion and therefore of priority inversion between equal-priority planes. Fourth, that genuine, non-determined multicore parallelism—not an interleaving chosen by a simulator but the modal fact that

*any* of several next steps may occur—is representable inside the same kernel as an internal box-and-diamond pair over the step alphabet, with trace equivalence induced by a commutation lemma between independent calls, and with platform modes as indices of a small Kripke frame in which an unbuildable mode yields *SKIP*, never a vacuous *PASS*. Every number quoted is produced by an executable harness: the assembled InterCore theory discharges 602 closed judgements spread over sixteen property groups, 15 of them named in the file for **Print Assumptions**, which reports no axioms, mirrored by 103 oracle checks over all  $8^6 = 262,144$  runs of six call sites, each site drawn independently from the eight operations admitted by the API; across the library—seven theories and eight oracles—the count is 944 judgements, and Section 8 states each of them.

**Keywords**—BTRON, Calculus of Inductive Constructions, equational reasoning, formal semantics, lock-free queues, InterCore, RTP, NuStreams, SMP, modal logic, model checking, theorem proving.

## 1 Introduction: The Model Is the Document

Operating-system specifications are written in a language designed for the purpose of being read, and are then implemented in a language designed for the purpose of being executed. The gap between the two is where real-time defects live. The BTRON cleanroom programme [2] has, from its first commits, maintained a third artefact: a mechanically checked definition of the subsystem under construction, written in the Rocq/Coq proof assistant [3,4], together with an executable oracle of the same definition in OCaml [5], run on every change by a single shell harness.

This paper argues a narrow methodological thesis with wide consequences: *for the classes of properties a real-time kernel actually needs, the whole of verification reduces to equational reasoning*, and the interesting engineering decisions are the ones that keep the equations in the reducing fragment. Not induction over traces, not bisimulation, not a separation logic of permissions—a calculus of rewriting totals over finite tables, closed by decidability of arithmetic. The scope is deliberately narrower than a whole-kernel verification [9, 13]: it is the subsystem boundary where BTRON’s real-time claims are actually made.

Four model families are treated:

1. the four *concurrency tiers* of the BTRON scheduling model—MP, AMP, SMP, Async (§3);
2. the *media* stack, from the NuStreams abstraction layer to RTP reception and the symmetric multi-plane pipeline (§4);
3. the **InterCore** protocol, the five-call message API between cores of the target four-core machine (§5);
4. the *modal extension* that lets one flat functional definition speak about genuinely non-determined parallel execution (§6).

Section 2 fixes the kernel and the five modelling idioms that recur in all four families. Section 7 describes the harness that keeps the mathematics and the machine honest, and states the one policy that makes the harness meaningful: *SKIP is not PASS*.

## 1.1 Why Equations, and Not Something Heavier

The properties demanded of a soft- and hard-real-time kernel are, with few exceptions, of four grammatical forms:

$$\begin{aligned}
 (\text{conservation}) \quad & n_{\text{in}} = n_{\text{out}} + n_{\text{held}} + n_{\text{lost}} \\
 (\text{monotonicity}) \quad & x(s') \leq x(s) \quad \text{or} \quad x(s) \leq x(s') \\
 (\text{invariance}) \quad & \pi_j(s') = \pi_j(s) \quad (j \neq i) \\
 (\text{provenance}) \quad & v = \pi_k(s') \Rightarrow v \in \pi_{\text{hist}}(s).
 \end{aligned} \tag{1}$$

Each of these is an *equation* between naturals or booleans computed from one state, or between two states related by one call. None of them needs a logic of resources. The consequence for the modelling language is severe and beneficial: the specification language can be the programming language, so long as the programming language is total.

## 1.2 Contributions

1. A single signature—total state transformer over a table-shaped bus, with derived accounting—is shown to cover all four concurrency tiers, the media stack, and InterCore (Section 3).

2. The watermark operator `min_fold` is identified as the shared algebraic content of multi-consumer retention in both media (§4) and InterCore (§5.4): residence, freeing, and the “no over-fill” theorem are the same three lemmas in two guises.
3. Cursor ownership is proved, not assumed, to be the mutex-elimination theorem of the InterCore star (§5.2), and the resulting absence of priority inversion between equal-priority INPUT and UI planes is stated as a corollary about the guards of the five calls.
4. Non-determined parallelism is internalised (§6) as a box/diamond pair over the step alphabet, computed by `forallb/existsb` on the same tables the model uses, with trace equivalence induced by commutation of independent calls—a machine-checked partial-order reduction.
5. An engineering account of what breaks when totality is taken seriously: the `Nat.sub` unfolding trap, the shadowing of `snd`, unary reduction of shipped constants (§2.6, §7).

## 2 The Equational Kernel

### 2.1 Judgemental Equality as the Only Discharge Rule

The CIC identifies propositions with types and proofs with programs; a *theorem* about a data structure is an inhabitant of a dependent product, and the overwhelming majority of inhabitants we construct are the single term *reflexivity*:

Listing 1: A frame lemma: writing one writer leaves the other writers untouched.

```

Lemma at_w_other_w : forall b i (w:writer) j, i <> j ->
  b_w (bus_w b i w) j = b_w b j.
Proof. reflexivity. Qed.

```

The point of *reflexivity* is that it checks *convertibility*—equality as computed by the kernel’s reduction machine—not equality as argued. When a lemma closes with *reflexivity*, the fact was already true by definition, and the specification has been arranged so that the fact is worth stating. The craft of the model consists in arranging for as much as possible to be definitional. Where computation is too shallow, we descend one step: `lia` for linear arithmetic over the successors and truncated subtractions that conversion leaves behind, and induction only where a fold genuinely recurses.

### 2.2 Totality, or the Absence of Admitted

Every function in the model is *total and terminating*, including the ones that the C implementation would signal as errors. The C `nth` is a partial function; the model’s

is `nth k 1 d`, which returns a default. The C ring buffer write on a full ring is undefined behaviour; the model’s returns a receipt. The discipline has three consequences we exploit throughout:

- there are no preconditions, hence no hypothesis book-keeping, hence no *vacuous-truth* bugs in the specification;
- every function is executable, so the identical text serves as the OCaml oracle (§7);
- the model cannot express “the kernel crashed here”, which is precisely the abstraction level wanted for a specification of a kernel that must not crash.

## 2.3 Tables: Finite Maps as Functions

The bus is a record of three *total* functions from dense small integers to cells, plus the one non-indexed coordinate:

```
Record bus : Type := mk_bus
{ b_cores : nat;
  (* cores that actually
    exist *)
  b_w, b_r, b_t : nat -> ... (* writers, readers,
    tasks *) }.
```

with update as a definitional three-place function:

```
Definition upd {A : Type} (t : nat -> A) (i : nat) (v :
  A) : nat -> A :=
  fun j => if Nat.eqb i j then v else t j.

Definition bus_w (b : bus) (i : nat) (w : writer) : bus
:=
  mk_bus (b_cores b) (upd (b_w b) i w) (b_r b) (b_t b).
```

Choosing `nat -> A` rather than a verified finite map (`PositiveMap`, a quotient, a `MapDom` typeclass) buys the entire frame-lemma algebra of Equation (1) (iii) at the cost of nothing: seventeen lemmas of the shape `at_w.same_w`, `at_r.bus_w`, `cores.inert.t` all close with `reflexivity` because the projections are definitions and the `if` is the only reduction needed. Dense integer indices are justified by the target: cursor identifiers in the shipped API are small integers into static tables (4 writers, 8 readers, 8 tasks, 64 sectors), and the C implementation on the ARM Cortex cores indexes arrays. Out-of-range indices are not excluded by types—they are *admitted* by boolean guards, which is the next idiom.

## 2.4 Derived Accounting, Never Stored Fields

A stored counter and the structure it counts can disagree; a derived quantity cannot. The model therefore stores history and cursors only, and defines the bookkeeping:

```
Definition w_enq (w : writer) : nat := length (w_hist w)
.
Definition r_deq (r : reader) : nat := r_cur r - r_start
r.
Definition held b (wid : nat) : nat :=
  if w_live (b_w b wid)
  then w_enq (b_w b wid) - floor_of b wid (w_enq (b_w b
    wid))
  else 0.
Definition total_held b : nat := sum_f (held b)
  max_writers.
```

```
Definition pool_free b : nat := sector_pool - total_held
  b.
```

$\text{published}(\text{wid}) = |\text{hist}_{\text{wid}}|$

$\text{consumed}(\text{rid}) = \text{cur}_{\text{rid}} - \text{start}_{\text{rid}}$

$\text{held}(\text{wid}) = \text{published}(\text{wid}) - \min_{\text{rid} \triangleright \text{wid}} \text{cur}_{\text{rid}}$

$\text{pool\_free} = 64 - \sum_{\text{wid}} \text{held}(\text{wid}).$

(2)

The conservation theorem of §5.6 is then a computation rather than an argument, and the class of states in which a counter was forgotten is not merely unprovable—it is uninhabitable.

## 2.5 Decide, Then Act

Every call is written as a cascade over *boolean* admission predicates, so that a proof may destructure the decision and keep the equation that explains the branch taken:

Listing 2: The five-line heart of `rcv`: three guards, three receipts, one state change.

```
Definition rcv (b : bus) (cl : caller) (rid : nat) : bus
  * rcv_result :=
  if negb (rcv_admit b cl rid) then (b, R_BAD_CURSOR)
  else if negb (rcv_ready b rid) then (b, R_EMPTY)
  else (bus_r b rid (reader_advance (b_r b rid)),
    R_DATA (nth (r_cur (b_r b rid)) (w_hist (b_w b (
      r_wid (b_r b rid)))) 0)).
```

The shape matters twice over. Operationally, it makes the “decide-then-act” seam visible: the state is changed *only* on the accepting branch, which is what makes the invariance lemmas of §5.5 hold by conversion. Logically, an `if` whose scrutinee has been deconstructed *with an equation label* (`eqn:R`) supplies the proof with exactly the hypothesis the branch’s value encodes—`R : rcv_admit b cl rid = true`—and the whole body then unfolds.

## 2.6 Two Traps Worth Recording

**Truncated subtraction under `cbn`.** `Nat.sub` is defined by recursion on its *first* argument. A goal such as `cur + 1 - start = S(cur - start)` is true, but `cbn` rewrites it into `match start with 0 ... S 1 ... end`, an expression `lia` treats as opaque—and reports *Cannot find witness*. The rule adopted across the theory: never normalise across a subtraction; instead rewrite accessor lemmas (`reader_advance_cur`, `reader_advance_start`) or restore linearity with `replace (S c - s) with (S (c - s)) by lia`.

**Name shadowing by the API.** The protocol contains a call named `snd`, which shadows the standard pair projection `Datatypes.snd`. Rather than rename the API—the API is the thing being specified—the kernel re-introduces the projection under its own name:

```
Definition pr2 {A B : Type} (p : A * B) : B := Datatypes
  .snd p.
```

It is a small humiliation, and a useful sign that the model is being read by the compiler rather than the author.

### 3 Four Tiers, One Signature

#### 3.1 The Signature

All four tiers are instances of:

$$\mathcal{M} = (S, s_0 \in S, \mathcal{A}, \delta : \mathcal{A} \rightarrow S \rightarrow S \times R, \pi_i : S \rightarrow \mathbb{N}), \quad (3)$$

with  $\mathcal{A}$  the alphabet of calls,  $R$  the receipts, and  $\pi_i$  the derived observables of Equation (2). A tier is then characterised by which coordinates  $\delta$  reads in its guard and which it writes in its state change.

The four instances, each read as alphabet, guard, effect, and determinism:

- **MP** — alphabet `pub`, `sub`; the admission predicate reads `b_w` and `b_t`, the accepting branch writes `b_w`; sequential by construction.
- **AMP** — adds `spawn`; the guard additionally reads `b_cores` and writes `b_t`; every task is pinned to a core.
- **SMP** — adds `snd`, `rcv`; the guard reads every coordinate together with `owns`, and the accepting branch writes `b_w` and `b_r`; the step relation is non-determined (§6).
- **Async** — `evt_put` over the event ring; head and tail each have exactly one writer; progress is bounded rather than determined.

#### 3.2 MP: The Uniprocessor Baseline

With `cores_shipped` = 1, the model degenerates honestly: `b_cores` = 1 makes `core <? b_cores b` admit only core 0, every `spawn` lands on core 0, and the star of Figure 1 has one spoke. This is not a limitation but a test of the specification: a model of multicore messaging that cannot be *run* on the shipped single-core board is not a model of the shipped board. The shipped-value section of the theory instantiates the general laws at `cores_shipped` and the OCaml oracle executes them.

#### 3.3 AMP: Asymmetric Tiers and the Core-as-Plane Map

In the asymmetric configuration, cores are not interchangeable: core 3 is the I/O core, and the INPUT/UI planes are pinned. The model expresses this by a single function

$$plane : core \Rightarrow Plane, \quad plane(3) = AsyncIO, \quad (4)$$

consulted by the `spawn` guard, whose full shape is:

```
Definition spawn_ok b (core : nat) (l : list cref) :
  bool :=
  andb (core <? b_cores b)
    (andb (negb (Nat.eqb (length l) 0))
      (andb (cursors_live b l)
        (andb (forallb (fun c => negb (
          cursor_taken b c)) l)
          (andb (Nat.leb (pub_count_c l)
            1) (star_ok b core l)))))).
```

Reading the conjuncts bottom-up gives the AMP discipline as one boolean: the request names a core that exists; the task arrives with work; every cursor named is live; no cursor named is already held; at most one publisher per task; and if the task brings a publisher, the target core does not already hold one. A failed conjunction leaves the bus *unchanged*, which is why `spawn` can be refused without rollback.

#### 3.4 SMP: One Queue Per Core, No Broker

The symmetric case is the interesting one, and its defining decision is negative: there is no shared multi-producer queue. Each core owns exactly one single-producer ring, and the multi-consumer view is computed:

$$MPSC(bus) \triangleq \lambda rid. hist_{wid(rid)}[cur_{rid}], \quad (5)$$

a pure projection, not a data structure. Symmetry is then the statement that the guard of `snd` reads only the writer cell of the calling core's queue and the global sector count, never another core's queue. The consequence is the parallel-composition law of §6.2.

The historical argument, from the SMP analysis [12], is that the Erlang/BEAM pattern [8]— $N$  threads bound to  $N$  cores each draining its own lock-free mailbox with a single `CMPXCHG8` on the producer tail—reaches message-passing throughput without kernel transitions. The equations above are that pattern with the machine instructions replaced by a functional projection, which is exactly what makes it provable.

#### 3.5 Async: Period, Budget, and the Masked Ring

The asynchronous tier contributes the temporal constants and the event queue:

```
Definition io_period_us : nat := 250.
Definition io_budget_us : nat := 500.
Definition event_queue_size : nat := 2 ^ 8.
```

With capacity  $2^8$ , head and tail are maintained as free-running counters and the slot index is obtained by masking,  $slot(n) = n \bmod 2^8$ . The equations that matter are the two mask laws, and they hold *because* the modulus is a power of two:

$$tail - head \leq 2^8 \quad head_1 \equiv head_2 [2^8] \Rightarrow slot(head_1) = slot(head_2). \quad (6)$$

Occupancy is  $tail - head$ , never  $tail - head$  computed in the wrong order—the single most common defect in ring buffer code, and here it is a type error, since the model has one occupancy expression and it is a difference of naturals bounded by Equation (6). One writer per field is the AMP assignment: the *consumer* owns *head*, the *producer* owns *tail*. Then the overflow policy is the interesting equation: drop-oldest, with *KEY* and *BUT*

events outside the droppable set, and *MOVE* coalescing expressed as a rewrite of the tail slot when its predecessor is a *MOVE* of the same device. The budget invariant is

$$T_{\text{drain}}(p) \leq \text{budget} = 2 \times \text{period}, \quad (7)$$

which permits one overloaded period to be absorbed without deadline loss—the reason the constant 500 is shipped rather than 250.

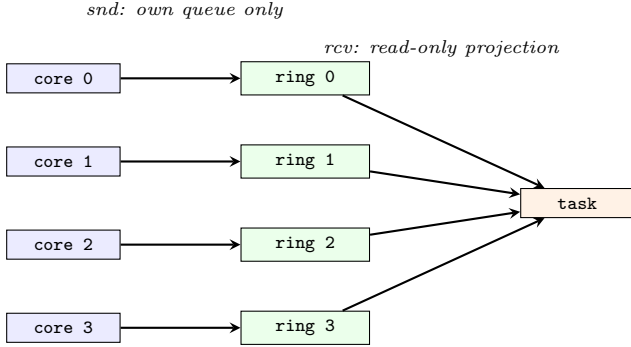


Figure 1: The InterCore star. Publishers write their own ring (left arrows, exclusive). Subscribers read through the derived projection  $\mathcal{M}$  (right arrows, pure). There is no node in the middle: no broker, no queue of queues, no kernel trap.

## 4 Media: Streams, RTP, and the Watermark

### 4.1 NuStreams as an Abstraction Adjunction

The NuStreams line of work on stream abstraction in process algebra treats a stream as an infinite object observed through a four-function interface: *val* reads a value, *err* reports an error, *abstr* maps a concrete stream into its abstraction, and *rep* reflects an abstract stream into a concrete one, with the fusion law

$$\begin{aligned} \text{val}_a(s) = v &\iff \text{val}_c(\text{abstr}^{-1}(v)) = v \\ \text{abstr} \circ \text{rep} &= \text{id} \end{aligned} \quad (8)$$

expressing that refinement preserves observable content. In the BTRON media model the adjunction is implemented by the **media** layer's typed ports: a port is a total function from a step index to a value cell, and **abstr** is a projection that forgets representation while **rep** re-materialises it. Because *val* is total in the model, the *err* component is a value, not an exception, matching Equation (3).

### 4.2 RTP Header Algebra

An RTP packet [10] is, in the model, a header algebra over  $\mathbb{Z}/2^{16}$  and  $\mathbb{Z}/2^{32}$ :

$$h = \langle pt, seq \bmod 2^{16}, ts \bmod 2^{32}, ssrc \rangle, \quad (9)$$

with the three laws that make a receiver correct. **(L1) Sequence continuity modulo wrap.** Define the ordered difference

$$\begin{aligned} \Delta(s_1, s_2) &= (s_2 - s_1) \bmod 2^{16} \\ s_1 \prec s_2 &\triangleq \Delta(s_1, s_2) < 2^{15}, \end{aligned} \quad (10)$$

so that  $\prec$  is the semicircle order, and the receiver's expected-sequence update is  $exp \leftarrow \Delta(exp, hold + buf)$ . The modelled invariant is that  $\prec$  is transitive on any window of width  $< 2^{15}$ , which is the condition under which reorder-by- $\Delta$  is a total order—and it is a computation in  $\mathbb{Z}/2^{16}$ , discharged by **lia** after replacing each modular difference by its representative.

**(L2) Timestamp-to-media-time.** For clock rate  $f$ ,  $\tau(n) = ts_n/f$  and inter-arrival jitter obeys the standard recursive estimate

$$J_{n+1} = J_n + \frac{1}{16} \left( |(a_{n+1} - a_n) - (ts_{n+1} - ts_n)/f| - J_n \right). \quad (11)$$

In an integral model the update is exact only up to truncation; the model states the bound  $|J_n - J'_n| \leq 1$  between the truncating and rounding variants, which is what a playout-buffer calculation needs, and it closes with **lia** after unfolding the division.

**(L3) Loss accounting is a conservation instance.** With  $E_n$  the extended sequence number,  $D$  the count of discarded-in-order packets,  $B$  the count still buffered,

$$\begin{aligned} E_n - E_0 &= R_n + L_n, & L_n &\geq D_n, \\ R_n &= \text{received}_n - B_n, \end{aligned} \quad (12)$$

and  $L_n$  is *defined* as the difference, never incremented separately. This is the derived-accounting idiom of §2.4 applied to statistics, and it is the reason the RTP theory has no “counter drift” lemmas: there are no counters to drift.

### 4.3 GStreamer as the Empirical Ground of the Axioms

The media theory's non-trivial content came from reading a production pipeline rather than a textbook. Against the GStreamer sources [11], the properties collected as the enrichment of **media\_rtp\_properties.v** encode the invariants its implementation relies on:

- Reference counting as a natural with a floor.** A buffer is alive while *refcount*  $> 0$ ; the model's **held** (§5.4) is exactly the multi-owner generalisation: residence ends when the *last* reader passes the item, so  $\text{refcount} = |\{rid : \text{cur}_{rid} \leq k\}| = 0$ .
- Segment/running-time.** GStreamer timestamps buffers in *running time* relative to a segment base,  $rt = ts - \text{base}$ , and every seek installs a new base. The model adopts the same split, which is what makes *start* immutable in **reader** while *cur* moves: *start* is the segment base, *cur* the running position.

3. **Caps negotiation is admission.** Pad capability intersection is an *admission predicate* returning a boolean, not an exception; the decide-then-act shape of §2.5 is the same discipline that keeps a pipeline from being left half-negotiated.
4. **Drop-oldest and the latency ceiling.** A live sink drops the oldest buffer when the queue exceeds its latency budget; the equivalent of `snd.fits` is the guard that turns a would-be overflow into a counted drop with a receipt (**(L3)** again).
5. **Coalescing.** Consecutive identical-typed events merge; the event-queue *MOVE* coalescing of §3 is this law, and both preserve FIFO order among survivors—an induction over the fold, not an axiom.

#### 4.4 The Symmetric Media Pipeline

The SMP media model (`media.smp.properties.v`) is the same watermark algebra with the plane index added: each plane  $p \in \{\text{INPUT}, \text{UI}, \text{DSP}, \text{IO}\}$  has its own ring, and cross-plane delivery is an *rcv* on the destination plane’s reader. The characteristic theorem is that a stalled UI plane cannot raise the *held* count of the INPUT ring beyond the ring capacity—a direct consequence of  $\text{held}(\text{wid}) \leq \text{w\_enq}(\text{wid})$  and of the watermark’s dependence on *readers of that queue only*.

### 5 The InterCore Calculus

#### 5.1 State

The three cell types, with the empty cell as a *value* rather than an absence:

```
Record writer : Type := mk_writer
{ w_live : bool; w_cap : nat; w_hist : list nat;
  w_drop : nat }.
Record reader : Type := mk_reader
{ r_live : bool; r_wid : nat; r_start : nat; r_cur :
  nat }.
Record task : Type := mk_task
{ t_live : bool; t_core : nat; t_curs : list cref }.
```

A cursor is either a publisher or a subscriber identifier, `cref := CPub wid | CSub rid`, and `task` holds a *list* of them. The task’s cursor list is the ownership store: there is no lock table, no kernel object, no reference count.

#### 5.2 Ownership: the Mutex-Elimination Theorem

```
Definition owns b (t : nat) (c : cref) : bool :=
  andb (t_live (b_t b t)) (existsb (cref_eqb c) (t_curs
    (b_t b t))).

Definition cursor_taken b (c : cref) : bool :=
  existsb (fun t => owns b t c) (nat_list max_tasks).

Definition usable b (cl : caller) (c : cref) : bool :=
  match cl with
  | NoTask => negb (cursor_taken b c)
```

```
| ByTask t => owns b t c
end.
```

The guard of `snd` and `rcv` is not “the queue is empty” or “the lock is free”; it is *usable*( $b, \text{caller}, c$ ). The five lemmas `usable_free`, `usable_taken`, `usable_owner`, `usable_stranger`, together with the inertness pair `usable_w_inert/usable_r_inert`, give the elimination result:

**Theorem 5.1** (Mutex elimination). *For every admitted call and every state, the set of cursors usable by a task is exactly the set it received from a successful `spawn`; no `snd`, `rcv`, `pub`, or `sub` alters usable; and `spawn` refuses any request naming a cursor already taken. Hence the sharing invariant is established once, at task creation, and the mutual-exclusion question is never asked again at run time.*

The real-time corollary is the one the shipping decision cares about: because no two equal-priority planes contend for a lock, there is no lock to be held by a low-priority context, and therefore *no priority inversion between INPUT and UI at equal priority*. A blocking primitive cannot give this guarantee; a non-blocking one plus exclusive ownership can, and the proof is the four lines of *usable*.

#### 5.3 The Five Primitives

The whole protocol is five functions, and the whole of their behaviour is four cell updates behind decidable guards. A ring is re-adopted, extended, charged with a loss, or a cursor is advanced; there is no fifth verb, and no update touches two cells. Writing  $w = (-, c, h, d)$  for a writer and  $r = (-, \text{wid}, s, \text{cur})$  for a reader:

$$\begin{aligned}
 \text{revive } \kappa w &= (\top, \kappa, h, d) \\
 \text{push } x w &= (\top, c, h \frown \langle x \rangle, d) \\
 \text{drop1 } w &= (\top, c, h, d + 1) \\
 \text{advance } r &= (\top, \text{wid}, s, \text{cur} + 1)
 \end{aligned} \tag{13}$$

which are `writer_revive`, `writer_push`, `writer_drop1` and `reader_advance` in the file. The five calls are short enough to quote entire, and every later judgement is a statement about these terms; the guards they test are defined immediately below the display, and the `first_dead` slot scans they call are one fuel-bounded search for an unused row:

```
Definition pub_slot (b : bus) : option nat :=
  first_dead (fun i => w_live (b_w b i)) max_writers 0.

Definition pub (b : bus) (cap : nat) : bus * option nat
:=
  match pub_slot b with
  | None => (b, None)
  | Some i => (bus_w b i (writer_revive (b_w b i) cap),
    Some i)
  end.

Definition sub_admit (b : bus) (cl : caller) (wid : nat)
: bool :=
  andb (w_live (b_w b wid)) (usable b cl (CPub wid)).
```

```

Definition sub (b : bus) (cl : caller) (wid : nat) : bus
  * option nat :=
  if sub_admit b cl wid then
    match sub_slot b with
    | None => (b, None)
    | Some i => (bus_r b i (mk_reader true wid
      (w_enq (b_w b wid)) (w_enq (b_w b wid))
      ), Some i)
  end
  else (b, None).

Definition spawn (b : bus) (core : nat) (l : list cref)
  : bus * option nat :=
  match spawn_slot b with
  | None => (b, None)
  | Some i => if spawn_ok b core l then (bus_t b i (
    task_revive core l), Some i)
    else (b, None)
  end.

Definition snd (b : bus) (cl : caller) (wid : nat) (d :
  nat) : bus * snd_result :=
  if negb (snd_admit b cl wid) then (b, S_BAD_CURSOR)
  else if negb (snd_fits b wid) then
    (bus_w b wid (writer_drop1 (b_w b wid)), S_FULL
    (S (w_drop (b_w b wid))))
  else (bus_w b wid (writer_push (b_w b wid) d), S_OK).

Definition rcv (b : bus) (cl : caller) (rid : nat) : bus
  * rcv_result :=
  if negb (rcv_admit b cl rid) then (b, R_BAD_CURSOR)
  else if negb (rcv_ready b rid) then (b, R_EMPTY)
  else (bus_r b rid (reader_advance (b_r b rid)),
    R_DATA (nth (r_cur (b_r b rid)) (w_hist (b_w b
    (r_wid (b_r b rid)))) 0)).

```

Three readings of that text carry the rest of the paper.

(i) *Refusal is stuttering.* Each guard is a closed boolean term over the bus, and for every call but one its negation returns the input state unchanged:

$$\gamma(p, b) = \text{false} \implies \text{call}(p, b) = (b, \rho), \quad (14)$$

with  $\rho$  the refusal receipt—`pub_no_slot_refuses`, `sub_full_refuses`, `spawn_refused_shape`, `snd_bad_cursor_inert`, `rcv_never_yields`. An error path is therefore inert by construction rather than by review, and a  $\square$ -validity survives a refusal because the refused step is the identity (§6.2). The exception is `snd`'s capacity guard, the one place where a negative branch is not the identity: a full ring charges the loss to its own cell and reports it, `snd_fits(b, w) = false` gives  $\pi_1(\text{snd } b \text{ cl } w \text{ d}) = b[w \mapsto \text{drop1}]$  (`snd_full_bus`), because a dropped sector is data the model must keep.

(ii) *Acceptance writes one cell of one table.* Writing  $b[X, i \mapsto v]$  for `bus_X b i v`, the whole frame discipline is one lemma about the underlying table update, `upd_other`,

$$i \neq j \implies (b[X, i \mapsto v])_X[j] = b_X[j], \quad (15)$$

instantiated as `at_w_other_w`, `at_r_other_r`, `at_t_other_t`, and strengthened across tables by the six lemmas `at_X bus_Y`, which say that writing table  $Y$  leaves table  $X \neq Y$  pointwise unchanged (§2.3). A writer cell is written by `pub`, which adopts a slot, and by `snd`, which extends one; a reader cell by `sub` and `rcv`; the task table by `spawn` alone. All four of the cursor-carrying calls pass the same *usable* test first, which is exactly where the ownership discipline of Theorem 5.1 bites.

Every stored coordinate is inert for the calls that do not write it, and the derived ones inherit it—`held_w_inert`, `usable_w_inert`, `floor_of_w_inert`—so 46 obligations in the file are named *inert*, *untouched* or *unchanged*. That number is not padding: each is a premise a later commutation step discharges by rewriting rather than by induction. Equation (15) is also what makes two calls to distinct cells commute, and commutation is the reduction that makes non-determined interleavings tractable (§6.3).

(iii) *Cost is bounded by the tables, not by the traffic.* Each call is one fuel-bounded scan of a fixed table plus one constructor application: nothing iterates over `w_hist`, nothing copies a list, nothing allocates. Hence the worst case of `rcv` is the three-way case analysis of §5.5 and the worst case of `snd` is two guards and a cons—independent of how deep the queue has become or how long the process has run. A full ring thus costs one word of state and cannot stall its caller: the accepted overflow path is `drop+1` and a receipt, not a wait.

There are three admission tests but two of them are the same term: `sub_admit` = `snd_admit`, both  $w\_live(b_w[b, w]) \wedge usable(b, cl, CPub\ w)$ . A core that may attach a subscriber to a ring is the core that may publish on it, and there is no second permission to keep consistent. Reading is gated by the cursor's own row and its publisher's,  $rcv\_admit(b, cl, \rho) = r\_live(b_r[b, \rho]) \wedge w\_live(b_w[b, r\_wid(b_r[b, \rho])]) \wedge usable(b, cl, CSub\ \rho)$ . What the last two calls add on top is arithmetic on derived counters,

$$\begin{aligned}
 \text{snd\_fits}(b, w) &= \text{held}(b, w) < w\_cap(b_w[b, w]) \\
 &\quad \wedge \text{total\_held}(b) < P \\
 \text{rcv\_ready}(b, \rho) &= r\_cur(b_r[b, \rho]) \\
 &\quad < w\_enq(b_w[b, r\_wid(b_r[b, \rho])])
 \end{aligned} \quad (16)$$

with *held* the watermark fold of §5.4 and  $P$  the sector pool. Nothing in Equations (13)–(16) is a specification *about* the implementation: they *are* the model that the C kernel is required to refine, and Section 8.2 states the headline judgements they yield, group by group.

## 5.4 Watermarks

Retention is computed, not tracked:

```

Fixpoint min_fold (f : nat -> bool) (g : nat -> nat) (
  fuel : nat) (i acc : nat) : nat :=
  match fuel with
  | 0 => acc
  | S k => min_fold f g k (S i) (if f i then Nat.min acc
    (g i) else acc)
  end.

Definition floor_of b (wid e : nat) : nat :=
  min_fold (fun r => sub_reader wid (b_r b r)) (fun r =>
    r_cur (b_r b r)) max_readers 0 e.

```

*floor\_of* is the fold, bounded by the seed  $e$ , of the cursors of the readers subscribed to `wid`. This single function is the shared content of §4 and §5: residence time, sector release, and the pool conservation law are three readings

of the same fold. Its four algebraic properties are what make it usable in proofs:

**Lemma 5.2** (Watermark algebra). *For all  $b, wid, e$ : (i)  $\text{floor\_of } b \text{ wid } e \leq e$ ; (ii)  $e_1 \leq e_2 \Rightarrow \text{floor\_of } b \text{ wid } e_1 \leq \text{floor\_of } b \text{ wid } e_2$ ; (iii) a join at the tail contributes  $\text{floor\_of } b \text{ wid } e = e$ , costing zero sectors; (iv) advancing a cursor can only raise the floor:  $\text{floor\_of } b \leq \text{floor\_of } (\text{bus\_r } b \text{ rid } (\text{advance } r))$ .*

Property (iii) is the statement that adding a subscriber to a live publisher costs nothing immediately—the new reader starts at the tail—and (iv) that reading is the only way sectors come back. Both are induction over `min_fold` with the congruence principle `min_fold_cong_if`, which encodes the semantic point that a fold need not respect the values of a function where the guard is false.

## 5.5 Receipts and the Shape Theorem

The two result types are the protocol’s entire error surface:

```
Inductive snd_result : Type := S_OK | S_FULL (dropped : nat) | S_BAD_CURSOR.
Inductive rcv_result : Type := R_DATA (v : nat) | R_EMPTY | R_BAD_CURSOR.
```

Note what is absent: no blocking outcome, no partial result, no error code requiring a retry loop. From the decide-then-act layout of §2.5 follow, for each call and each receipt constructor, a *shape* lemma naming the branch, a *bus* lemma pinning the state change, and a *receipt* lemma computing the payload—the pattern

```
Lemma rcv_data_bus : forall b cl rid (D : rcv_ready b rid = true),
  fst (rcv b cl rid) = bus_r b rid (reader_advance (b_r b rid)).
```

The receipt kit—nine lemmas for `rcv` alone, a shape, bus, and receipt lemma for each of its three outcomes—is the reason the remaining obligations have hypotheses instead of case analyses: destructuring the receipt substitutes the guard equation, and the guard equation unfolds the definition.

## 5.6 Conservation

**Theorem 5.3** (Conservation of items and sectors). *For every admitted call: (a)  $w\_enq$  is monotone and increases by at most one; (b) a dropped item is counted, never lost:  $w\_drop$  increases by one and  $w\_hist$  is unchanged; (c)  $\sum_{wid} \text{held}(wid) \leq \max_{writers} \times \text{ring\_capacity} = 16 < 64 = \text{sector\_pool}$ ; (d) the pool balances:  $\text{pool\_free}(s') + \text{total\_held}(s') = \text{pool\_free}(s) + \text{total\_held}(s)$  for every call; (e) a *snd* whose cursor sits at the tail is read back verbatim by the corresponding *rcv*: received appends exactly the item published.*

Statement (c) is why `pool_room` is a redundant guard in the shipped configuration—kept for the general case,

proved inert here. Statement (d) is the reason the launch-time accounting of §7 closes: sectors leave the pool only through `held`, and every path that raises `held` lowers `pool_free` by the same amount, by conversion. Statement (e) is the broker-freeness of Equation (5) observed at the level of values rather than state: the item crosses the star, and the receipt names it.

## 5.7 The Sixteen Property Groups

The theory is organised in sixteen groups, each a section of the same file and each mirrored by an identically named group in the OCaml oracle. Section 8.2 states the headline judgement of every group classically, with the vocabulary it is written in and the reason it was worth proving; the counts below are the closed judgements currently discharged in each group’s sections, so that the shape of the effort is visible before its content. The model layer beneath the sixteen groups — the frame lemmas of §2.3, the derived accounting, the receipts, and `bus_wf` — carries a further 67, for 602 in the file.

- I1 Admission** (16): slot searches terminate inside the static table, a full table refuses with a receipt, grants are distinct, and no call creates a core.
- I2 Topology** (67): a subscriber joins at its publisher’s tail and pins nothing; `snd` respects capacity and pool, and never over-fills.
- I3 Non-blocking** (48): the three receipts are exclusive and exhaustive; `rcv` never yields, never rewinds, never advances by two, never spends a slot when empty.
- I4 Integrity** (10): `received` is one slice of the publisher’s own history, its items were published, and a read appends.
- I5 Star** (20): one publisher per task, one publisher-holding task per core, watermark locality, and a characterisation of `star_ok`.
- I6 Naming** (37): `spawn` validates `core_id`; each refusal leaves the bus untouched; an admission is exactly the request.
- I7 Ownership** (28): one cursor, one owner, preserved by all five calls (Theorem 5.1).
- I8 Conservation** (19):  $enq = deq + drop + held$  (Theorem 5.3), and quiescence returns the whole pool.
- I9 Determinacy** (52): `bus_wf` is an inductive invariant — the general law standing in for the exhaustive interleaving search.
- I10 Planes** (21): mode-indexed supply and budget equations, with `SKIP` for an unbuildable mode.
- I11 Events** (56): masked power-of-two ring algebra, one writer per field, FIFO across the wrap, proved for a 600-step soak.
- I12 Commutation** (48): two independent grants in either order give observably equal buses, and the class is closed under that equality.

- I13 *Modality* (35): box and diamond over the step alphabet with K, duality and the **always** induction rule; **bus\_wf** proved always.
- I14 *Branching* (35): witnesses that the enabled set is not a singleton and that call order is observable.
- I15 *Visibility* (23): the subscriber cursor is a floor; a store is in the fabric while only the publish is visible.
- I16 *Catch-up* (20):  $\text{pending}(\text{read\_k } k) = \max(0, \text{pending} - k)$ , and a drain stops at the tail.

## 6 Modal Extensions: Non-Determined Parallelism Inside the Kernel

### 6.1 The Problem with Interleaving Semantics

An interleaving semantics models concurrency as a non-deterministic choice among next steps. It is sound and it is what implementers test, but as *specification* it smuggles in a scheduler: the choice is made by the semantics, hence by the prover, hence—in practice—by the enumeration order of a checker. What the multicore BTRON target needs asserted is the stronger, modal statement: *whatever* the hardware chooses next, these equations survive. Determinism is not the goal; *invariance under non-determinacy* is.

### 6.2 Runs, and Two Internal Modal Operators

A state is  $s \in S$ ; a step is  $a \in \mathcal{A}$ ; with  $\delta$  total, a *run* is simply a finite word  $w \in \mathcal{A}^*$  acting by fold:

$$s \cdot \epsilon = s, \quad s \cdot (w \smallfrown \langle a \rangle) = \pi_1(\delta(a, s \cdot w)). \quad (17)$$

Reachability is then a predicate on words, and the two modalities are its quantifier-free avatars:

$$\Box_k \varphi \triangleq \forall w \in \mathcal{A}^k. \text{holds}(\varphi, s \cdot w) \quad (18)$$

$$\Diamond_k \varphi \triangleq \exists w \in \mathcal{A}^k. \text{holds}(\varphi, s \cdot w). \quad (19)$$

Crucially, both are *computable* in the model, because they are folds over lists of naturals: **forallb** and **existsb** already appear in the guards of Theorem 5.1. *core\_holds\_publisher* is a diamond ( $\Diamond_1$ : some live task holds a publisher on this core); **spawn\_ok**'s **forallb** (**fun c => negb (cursor\_taken b c)**) is a box ( $\Box_{|U|}$ : every named cursor is free). The modal extension therefore does not enrich the logic; it reads the modality off definitions that were already needed.

The frame validated by these operators is the standard one. Let  $R \subseteq S \times S$  be one-step reachability.

- $R$  is reflexive where a call is refused: **spawn\_ok** false yields  $(b, \text{None})$ , i.e.  $sRs$ . A step that may be refused

makes *no* progress claim; every  $\Box$ -validity is preserved by stuttering, which is why refusals are modelled as identity rather than error.

- $R$  is *not* confluent, and the theory says so: two **snd** calls from different cores to the same ring, in the two orders, give different *hist* appendices. Non-determinacy is real; the invariants proved are exactly those insensitive to it.
- Termination of each step is by totality, so the box over *some*  $k$  is decidable: with  $|S|$  effectively bounded by the table sizes and the run length chosen by the harness,  $\Box_k$  reduces to a finite fold—the oracle of §7.

### 6.3 Independence and Trace Equivalence

The tool that converts a proof about all interleavings into proofs about two is commutation. For calls  $a, b$  on disjoint coordinates:

$$\delta(a, \pi_1(\delta(b, s))) = \delta(b, \pi_1(\delta(a, s))) \quad (a \perp b), \quad (20)$$

and the same for observables  $\pi_i \circ \delta(a) \circ \delta(b) = \pi_i \circ \delta(b) \circ \delta(a)$  when  $\pi_i$  reads a third coordinate. The invariance lemmas of §2.3—at\_w\_other\_w, at\_r\_bus\_w, cores\_inert\_t, and their relatives, seventeen in all—are exactly the decomposition of Equation (20) into its coordinate projections. Discharging **reflexivity** seventeen times is a partial-order reduction that no model checker needs to be told about: the quotient by the commutation relation (a Mazurkiewicz trace monoid [7]) is available definitionally, so the state space collapses *before* any search.

Concretely, InterCore admits: any two **snd** to *distinct* live rings commute; any **rcv** commutes with any **snd** to a ring the reader does not pin; **pub/sub/spawn** commute with everything that touches no cell in their table. The non-commuting residue—same-ring writes, and a **rcv** racing its own **snd**—is small enough that the  $\Box$  claim over it is what I9 verifies by exhaustion.

### 6.4 Modality of the Platform, and SKIP

The second modal axis is the build configuration, which behaves like an index set for the validity relation, i.e. a small Kripke frame over modes:

$$m \in \{\text{MP, AMP, SMP, planes, cores=N}\}, \quad (21)$$

$$m \Vdash \varphi \triangleq \text{verify\_models.sh accepts } \varphi \text{ under } m.$$

The shipping contract fixes the accessibility relation: with **BTRON\_MP=0** the queue-mode obligations are *not* valid and are not silently assumed valid—they are reported **SKIP**. The **--mode=planes** configuration is the Tier-1 contract, and **--cores=N** is a *request*: oversubscription above **cores\_target** is an error, not a degradation. The modal content is that the general laws of §5 are valid at every mode, whereas the numeric checks of §7 are valid only where the mode makes them meaningful. A theory whose

theorems are mode-indexed rather than mode-independent is a test suite wearing a hat; ours keeps the theorems unconditional and pushes the indexing into the harness.

## 6.5 Progress Without Determinism

The last modal property is the one a real-time reader wants. `rcv` is *strictly* non-blocking: there is no outcome in which the call parks the caller, and the three receipts exhaust the outcome space (`rcv_receipt_exhaustive`). Therefore every step has bounded duration and

$$\forall s. \forall a \in \mathcal{A}. \Diamond_1 \text{settled}(a, s), \quad (22)$$

i.e. under every resolution of the non-determined choice, the call terminates with a receipt. Combined with Theorem 5.1 (no lock can be held against you) and Theorem 5.3(d) (sectors are conserved, so a full pool is always attributable to unread readers), this is the determinacy half of the multi-core real-time argument: the schedule is not determined, but its effects on the invariants are.

## 7 Two Oracles, One Text

### 7.1 Proof and Execution From the Same Definitions

The OCaml model file is not a translation of the Coq model; it is the same term, in a syntax that happens to compile the same function bodies. Two oracles then do different jobs. The Coq theory proves the *general laws* (all states, all calls, all interleavings, by construction). The OCaml oracle checks the *shipped constants* and searches  $\square_k$  exhaustively:

```
$ ./media_intercore_model
==> B-System media_intercore model: pub/sub/spawn/snd/
rcv cursors,
star topology, sector pool
PASS: OCaml oracle (media_intercore_model.ml) -
InterCore invariants I1..I16 passed (103 checks,
262144 interleavings).
$ coqc media_intercore_properties.v # silent,
exit 0: nothing left open
$ coqchk -o -silent media_intercore_properties
* Axioms: <none>
* Constants/Inductives relying on type-in-type: <none>
* Constants/Inductives relying on unsafe (co)fixpoints:
<none>
* Inductives whose positivity is assumed: <none>
```

The search is eight call forms at each of six positions,  $8^6 = 262,144$  runs from a seeded bus, and the oracle records which *interesting* branch was reached during the search—so that “sectors really leave the pool under interleaving” and “a full ring really reports *S\_FULL*” are witnessed, not merely unrefuted. Coverage witnesses are what distinguishes an exhaustive search from an exhaustive pass.

### 7.2 Where the Literal Numbers Belong

The launch accounting of §7.4 of the verification plan fixes target figures such as ring publishes and dequeues in the  $10^6$

range with a residual drop count and *pool\_free* restored to its initial value. Those arithmetic facts are asserted *in the oracle*, not in the theory, for a mechanical reason: numerals of that size enter CIC as `Nat.of_num_uint` wrappers that `lia` cannot see through, and `vm.compute` on unary naturals is not a strategy. The division of labour is principled rather than convenient—*general laws in Coq, shipped constants in OCaml*—and it is why `ring_capacity`, `sector_pool`, and `event_queue_size` appear in the theory as names rather than literals, so the same proofs instantiate at four rings of four slots today and at any shipped value later.

### 7.3 Print Assumptions: Axiom Hygiene

A total, computational model can be kept axiom-free, and doing so constrains the modelling vocabulary productively:

- *No functional extensionality.* Every invariance statement is pointwise ( $\forall j, i \neq j \Rightarrow \pi_j(s') = \pi_j(s)$ ) or convertible, so the seventeen frame lemmas of §2.3 never need to promote pointwise equality to function equality.
- *No quotient types, no classically choice-flavoured axioms.* Slot allocation is `first_dead` with an explicit fuel parameter—a decidable search returning `option nat`, whose specification (a `Some` result is in range and names a dead cell; a `None` result means no cell in the fuel is dead) is four lemmas proved by induction on the fuel.
- *No excluded middle for admission.* Guards are boolean functions and the proofs use `andb_true_iff`, `Nat.eqb_eq/Nat.eqb_neq`, `Nat.ltb_lt`—constructive decidability, which is exactly the modal content of §6.4.

The harness does not take `coqc`’s word for it: after each theory compiles, `coqchk -o -silent` re-validates the `.vo` in the kernel, and the four lists it prints—axioms, type-in-type dependencies, unsafe (co)fixpoints, assumed positivity—must each read `<none>`. A non-empty list fails the build even if every obligation closed, and the summary count distinguishes the failure from a `SKIP`. This is the difference between a proof and a proof modulo wishes.

### 7.4 SKIP Is Not PASS

The policy is one line in the harness, and it is the line that makes the other sections trustworthy: an obligation whose mode is not compiled in is reported `SKIP`, is counted separately from `PASS`, and is printed at the end. A suite of sixteen registered entries—eight oracles and eight theory files, one of which the tree does not yet contain—whose Coq array silently tolerated an unbuildable file reports a property of the shell script, not of the kernel. Related discipline, learned the hard way in this programme: a `coqc` invocation piped through `head` loses its exit status through the pipe; a missing diagonal row in the monitor can mean the flag never compiled in rather than the code being wrong; and an assertion that cannot be built is marked `SKIP`, never optimistically `PASS`.

## 7.5 Reading the Harness Output as a Specification

The harness emits one verdict per model—OCaml oracle and Coq theory separately—and a summary count that is the only number the reviewer is asked to trust. The media block of a current run:

```
==> OCaml model (media_intercore_model.ml)
PASS: OCaml oracle (media_intercore_model.ml) -
      InterCore invariants
      I1..I16 passed (103 checks, 262144 interleavings).
==> OCaml model (media_rtp_model.ml)
PASS: OCaml oracle (media_rtp_model.ml) - media
      invariants R1..R10 passed (80 checks).
==> OCaml model (media_smp_model.ml)
      ring: explored 279936 interleavings (6 ops ^ length
      7)
PASS: OCaml oracle (media_smp_model.ml) - invariants S1
      ..S11 passed
      (52 checks, 279936 interleavings, 35 cursor
      triples).
==> Coq/Rocq properties (media_intercore_properties.v)
PASS: coqc + coqchk (media_intercore_properties.v -
      theorems closed, no axioms)
==> Coq/Rocq properties (media_properties.v)
FAIL: coqc exited non-zero for media_properties.v
Error: Can't find file ./media_properties.v
==> Summary: 15 passed, 1 failed
```

The last three lines are quoted deliberately. `media_properties.v` — the global model of §9 — is registered in the harness array and has not been written, so the run reports **FAIL** and the summary reads 15 passed, 1 failed, not a clean sheet. A suite that printed only its passes would be a marketing artefact; this one is a specification, including about its own gaps.

Each line is a claim about a *definition* that the reader may open and read. Nothing here is a claim about timing, memory, or the compiler—those are the C implementation’s burden, discharged by instrumentation on the board. The division is deliberate: mathematics owns the invariants, hardware owns the constants.

## 8 The Coq Base Library: A Theorem Accounting

Sections 2–6 argue method; this section is the ledger, item by item. The mathematics lives in seven COQ files under `verify/models/`, one per subsystem, each with its OCaml oracle beside it and both registered in the same harness array. What follows is stated in the classical form — the *definition* a claim is written in, the *judgement* as the file states it, and the *remark* that says why the judgement was worth its price in proof. No proof is reproduced: a proof in this corpus is **reflexivity**, **lia**, or an induction of a few lines, and the interesting artefact is the statement.

**Counting rule.** Obligations are the closed judgements introduced by **Lemma**, **Theorem**, **Corollary** and **Example**; vocabulary is introduced by **Definition**, **Record**, **Inductive**, **Fixpoint** and **Notation**. Both counts are keyword counts over the file, and the line counts are `wc -l`, so the inventory below is reproducible from

the tree by `grep`. The two numbers that decide soundness are not counted but asserted: **admit** and **Admitted** occur nowhere in the corpus except in prose, and **Axiom**, **Parameter** and **Hypothesis** occur nowhere at all. Every obligation in every file is closed, because one open obligation would make `coqc` emit a warning the harness prints and `coqchk -o` name in its axiom list.

### 8.1 Inventory

The library as it stands, seven theories and eight oracles under one harness:

- `media_intercore_properties.v` — the InterCore star and the modal layer: 6,217 lines, 127 vocabulary declarations, 602 closed judgements, 15 **Print Assumptions** lines, sixteen groups I1–I16 (§8.2).
- `media_rtp_properties.v` — NuStreams, RTP, budgets, the fork rule: 1,976 lines, 107 declarations, 161 judgements, 12 gate lines, ten groups R1–R10 (§8.3).
- `media_smp_properties.v` — the CAS ring, AMP ownership, the seqlock: 698 lines, 39 declarations, 69 judgements, eight groups (§8.4).
- `hypermedia_dnd_properties.v` — Real/Virtual Body, the direct-manipulation protocol, TAD persistence: 1,033 lines, 55 declarations, 61 judgements, 6 gate lines (§8.5).
- `bfs_properties.v` — the Volume V2 journal and its write-ahead log: 553 lines, 21 declarations, 39 judgements, 6 gate lines; with `bfs_btree_properties.v` (191 lines, 10/6) and `bfs_allocator_properties.v` (140 lines, 11/6) (§8.6).

That is 10,808 lines, 370 vocabulary declarations and 944 closed judgements, of which 39 are additionally named in the file for a reader who wants to ask the kernel about them directly. The distribution is deliberately uneven. InterCore carries 602 of the 944 obligations because it is the only subsystem whose claims are about *non-determined* execution, and a modal property decomposes: one  $\square$  claim is a reachability lemma, an induction rule, and a corollary per conjunct. The file-system theories are small because their claims are local to one data structure. Three files (`bfs_btree`, `bfs_allocator`, `media_smp`) predate the in-file gate and are audited by `coqchk` alone; the gate block is now the convention, and adding it to those three is the cheapest outstanding work in the library.

### 8.2 The InterCore Theory

Sixteen groups, 535 obligations, standing on a 67-obligation model layer, and mirrored line for line by the identically numbered groups of `media_intercore_model.ml`, which runs 103 checks over the same alphabet. The vocabulary is the one fixed in §5; it is repeated here only so that each statement below can be read on its own.

**Definition 8.1** (Bus, cursors, derived holding).

```

Record writer := mk_writer { w_live : bool; w_cap : nat
                           ; w_hist : list nat; w_drop : nat }.
Record reader := mk_reader { r_live : bool; r_wid : nat
                           ; r_start : nat; r_cur : nat }.
Record bus    := mk_bus { b_cores : nat; b_w : nat -> writer
                        ; b_r : nat -> reader; b_t : nat ->
                           task }.
Definition w_enq (w : writer) : nat := length (w_hist w).
Definition held b (wid : nat) : nat :=
  if w_live (b_w b wid)
  then w_enq (b_w b wid) - floor_of b wid (w_enq (b_w b wid)) else
  0.
Definition bus_wf b := wf_tables b /\ wf_bounded b /\
  wf_reader_bounds b
  /\ wf_tasks b /\ wf_ownership b /\ wf_star b /\ wf_retired b.

```

*Remark 8.1.* Counters that are stored can disagree; these cannot. *w\_enq* is a length, *held* a difference of two cursors, *bus\_wf* the one conjunction every group below reads as its hypothesis. The defect class “the bookkeeping said one thing and the data said another” is not proved absent, it is unnameable.

**I1 — Admission: the tables are static****Definition 8.2** (Slot search). *Fixpoint* first\_dead (slot :

```

  nat -> bool) (fuel i : nat) : option nat :=
  match fuel with
  | 0   => None
  | S k => if negb (slot i) then Some i else first_dead slot k (S
    i)
  end.
Definition pub_slot (b : bus) : option nat :=
  first_dead (fun i => w_live (b_w b i)) max_writers 0.

```

**Theorem 8.1** (A full table refuses; a grant is new).

*Lemma pub\_full\_refuses* : *forall* b cap,  
 (*forall* j, j < max\_writers -> w\_live (b\_w b j) = true) -> pub b  
 cap = (b, None).  
*Lemma pub\_grants\_are\_distinct* : *forall* b cl i j,  
 pub\_slot b = Some i -> pub\_slot (fst (pub b cl)) = Some j -> i < j.

*Remark 8.2.* Post-boot allocation is not forbidden by policy, which a code path can bypass; it is inexpressible, because the only way to name a writer is to point at a cell of a fixed table. An exhausted table returns *None*, so the worst outcome of over-subscription is an error code at the call site rather than memory pressure somewhere else.

**I2 — Topology: a join lands at the tail****Theorem 8.2** (Late subscribers cannot replay).

*Lemma sub\_joins\_at\_the\_tail* : *forall* b cl wid i,  
 sub\_slot b = Some i -> sub\_admit b cl wid = true ->  
 r\_start (b\_r (fst (sub b cl wid)) i) = w\_enq (b\_w b wid) /\  
 r\_cur (b\_r (fst (sub b cl wid)) i) = w\_enq (b\_w b wid) /\  
 r\_wid (b\_r (fst (sub b cl wid)) i) = wid.

*Remark 8.3.* A subscriber seeds at its publisher’s tail and therefore holds zero sectors at birth. This is the memory claim of the whole layer: retention is bounded by the tables and the ring capacities, never by how long a consumer has been absent, and a daemon that restarts cannot pin a week of history.

**I3 — rcv is total and strictly non-blocking****Definition 8.3** (The protocol’s whole error surface).

*Inductive* snd\_result := S\_OK | S\_FULL (dropped : nat) |  
 S\_BAD\_CURSOR.  
*Inductive* rcv\_result := R\_DATA (v : nat) | R\_EMPTY | R\_BAD\_CURSOR.

**Theorem 8.3** (The receipts are exhaustive).

*Lemma rcv\_receipt\_exhaustive* : *forall* b cl rid,  
 pr2 (rcv b cl rid) = R\_BAD\_CURSOR  $\vee$  pr2 (rcv b cl rid) = R\_EMPTY  
 $\vee$  (*exists* d, pr2 (rcv b cl rid) = R\_DATA d).

*Remark 8.4.* There is no inhabitant of the result type meaning “park the caller”, so *rcv* cannot become an implicit scheduler yield; its worst case is a three-way case analysis. The companion lemmas of the group add the other half of the real-time argument: the three constructors are exclusive, a read never rewinds a cursor, never advances it by two, and never spends a slot when the publisher has nothing.

**I4 — Integrity of what was received****Definition 8.4** (The receipt, derived).

*Definition* received b (r : reader) : list nat :=  
 firstn (r\_deq r) (skipn (r\_start r) (w\_hist (b\_w b (r\_wid r)))).

**Theorem 8.4** (Every receipt is a slice of its own publisher).

*Lemma received\_is\_a\_slice* : *forall* b r n, r\_deq r = n ->  
 received b r = firstn n (skipn (r\_start r) (w\_hist (b\_w b (r\_wid r)))).

*Remark 8.5.* Torn reads, replay, and cross-talk between arms of the star are the three ways a lock-free multi-consumer queue is normally wrong, and here they are excluded by the shape of the definition: a reader’s history *is* a contiguous slice of one writer’s *w\_hist*. A value an arm never published cannot be read by it, because there is no other place for it to come from.

**I5 — The star is the topology****Definition 8.5** (One publisher per core).

*Definition* star\_ok b (core : nat) (l : list cref) : bool :=  
 if negb (Nat.eqb (pub\_count\_c l) 0)  
 then negb (core\_holds\_publisher b core) else true.

**Theorem 8.5** (A core has at most one publisher). *Lemma*

*publisher\_per\_core\_is\_unique* : *forall* b t1 t2, bus\_wf b ->  
 holds\_pub b t1 = true -> holds\_pub b t2 = true ->  
 t\_core (b\_t b t1) = t\_core (b\_t b t2) -> t1 = t2.

*Remark 8.6.* The broker does not exist, so it cannot be the bottleneck, the latency source, or the single point of failure; and two tasks on one core cannot both own the same publisher cursor. This is the theorem that lets the multi-consumer view be a projection (Equation (5)) rather than a component.

**I6 — spawn validates before it grants**

**Definition 8.6** (Admission).

```

Definition spawn_ok b (core : nat) (l : list cref) : bool :=
  andb (core <? b_cores b)
    (andb (negb (Nat.eqb (length l) 0))
      (andb (cursors_live b l)
        (andb (forallb (fun c => negb (cursor_taken b c)
          (andb (Nat.leb (pub_count_c l) 1) (
            star_ok b core l)))))).

```

**Theorem 8.6** (An admission is exactly the request).

*Lemma* spawn\_grants\_ownership : **forall** b core l i c,  
 spawn\_slot b = Some i → spawn\_ok b core l = true → In c l →  
 owns (fst (spawn b core l)) i c = true.

*Remark 8.7.* Half-granted ownership — a cursor left owned by nobody, or by two tasks, after a rejected call — is the classic error-path bug, and the group kills it in both directions: `spawn_grants_ownership` for the admitted case and six refusal lemmas, one per conjunct of `spawn_ok`, each proving the returned bus is the input bus. The error path is as safe as the success path.

**I7 — Ownership: the rule that removes the mutex****Definition 8.7** (The caller *is* the lock).

```

Definition owns b (t : nat) (c : cref) : bool :=
  andb (t_live (b_t b t)) (existsb (cref_eqb c) (t_curs (b_t b t))
    ).
Definition usable b (cl : caller) (c : cref) : bool :=
  match cl with
  | NoTask => negb (cursor_taken b c)
  | ByTask t => owns b t c
  end.

```

**Theorem 8.7** (Exclusivity).

*Lemma* usable\_is\_exclusive : **forall** b t1 t2 c, bus\_wf b →  
 usable b (ByTask t1) c = true → usable b (ByTask t2) c = true →  
 t1 = t2.

*Remark 8.8.* One cursor, one owner, definitionally — and since no two equal-priority planes contend for a lock, there is no lock a low-priority context can hold, hence no priority inversion between the INPUT and UI planes (Theorem 5.1). The remaining 27 obligations of the group are the other half: the invariant survives all five calls, so the question is asked once at task creation and never again at run time.

**I8 — Conservation of sectors****Theorem 8.8** (The gap is the holding, seen from any subscriber).

*Lemma* conservation\_at\_the\_floor : **forall** b wid rid,  
 rid < max\_readers → r\_live (b\_r b rid) = true → r\_wid (b\_r b  
 rid) = wid →  
 w\_live (b\_w b wid) = true →  
 floor\_of b wid (w\_eng (b\_w b wid)) = r\_cur (b\_r b rid) →  
 r\_cur (b\_r b rid) + held b wid = w\_eng (b\_w b wid).  
*Lemma* pool\_never\_overdrawn : **forall** b, bus\_wf b → total\_held b <= sector\_pool.

*Remark 8.9.* This is the leak assertion of the launch review, as a theorem:  $eng = deq + drop + held$  coordinate by coordinate, the pool is never overdrawn, and the quiescence lemmas give  $pool\_free = sector\_pool$  once every subscriber has caught up. A soak therefore proves conservation instead of hoping nothing leaked.

**I9 — The class the five calls cannot leave****Definition 8.8** (Steps and runs).

```

Definition apply_step (b : bus) (st : step) : bus :=
  match st with
  | Pub_step cap => fst (pub b cap)
  | Sub_step wid => fst (sub b NoTask wid)
  | Snd_step wid d => fst (snd b NoTask wid d)
  | Rcv_step rid => fst (rcv b NoTask rid)
  | Spawn_step core l => fst (spawn b core l)
  end.
Fixpoint run (b : bus) (steps : list step) : bus :=
  match steps with
  | nil => b
  | st :: tl => run (apply_step b st) tl
  end.

```

**Theorem 8.9** (Inductivity over all words).

*Theorem* interleavings\_preserve\_bus\_wf : **forall** steps,  
 (**forall** st, In st steps → wf\_step st) → bus\_wf (run bus0 steps)  
 .

*Remark 8.10.* The general law that the oracle’s  $8^6 = 262,144$ -run search can only witness. Its six corollaries — no run overdraws the pool, exceeds the rings, over-fills a ring, shares a cursor, has two publishers on a core, or changes the core count — are what turn “no counterexample found” into a proof, and they are the reason the enumeration below is a coverage instrument rather than the argument itself.

**I10 — Modes, and a gate that cannot fake coverage****Definition 8.9** (Verdicts).

```

Inductive verdict := V_PASS | V_SKIP | V_FAIL.
Definition row_verdict (mp : mp_flag) (cores : nat) (require :
  bool) (r : vrow) :=
  if negb (row_compiled mp r) then V_SKIP
  else if row_has_the_topology r cores then V_PASS
  else if require then V_FAIL else V_SKIP.

```

**Theorem 8.10** (The target topology passes every row; the shipped port skips one).

*Theorem* the\_target\_topology\_passes\_every\_row :  
**forall** r, row\_verdict MP\_ON cores\_target true r = V\_PASS.  
*Theorem* shipped\_port\_four\_pass\_one\_skip : run\_summary MP\_ON  
 cores\_shipped false  
 = (4, (1, 0)).  
*Theorem* no\_gate\_ever\_reports\_an\_uncompiled\_row : **forall** mp cores  
 require r,  
 row\_verdict mp cores require r = V\_PASS → row\_compiled mp r =  
 true.

*Remark 8.11.* SKIP is a value in the result type, and the second theorem is the harness’s policy as mathematics: on the one-core port four of the five rows pass, one reports V\_SKIP, and the V\_FAIL count is zero — a row that is not compiled in cannot be counted as covered. The third is the general form of the same claim, quantified over gates, core counts and rows, and it is the theorem that makes a faked pass in the shell script a contradiction rather than an overnight surprise.

**I11 — The event queue: masked indices, one writer per field**

**Definition 8.10** (A power-of-two ring whose cursors are never wrapped).

**Record** evtq := mk\_evtq { q\_buf : nat -> option nat  
; q\_head : nat; q\_tail : nat }.

**Definition** evt\_idx (k : nat) : nat := k mod event\_queue\_size.

**Definition** occupancy (e : evtq) : nat := q\_tail e - q\_head e.

**Theorem 8.11** (The wrap law, and a 600-step soak in order). *Theorem* evt\_idx\_wraps : forall k, evt\_idx (k + event\_queue\_size) = evt\_idx k.

*Theorem* soak\_of\_six\_hundred\_is\_fifo : rev (pr2 (soak 600 1 evtq nil)) = seq 1 600.

**Remark 8.12.** The wrap is the one place a ring defect survives testing silently, because a short test never reaches it: 256 steps must be run, and *evt\_idx* must be proved periodic rather than sampled. *soak* is the executable producer/consumer loop, and its FIFO law closes by computation — the same computation the oracle runs, which is why the two agree without a bridge lemma.

## I12 — What two cores can and cannot tell about each other

**Definition 8.11** (Observational equality).

**Definition** obs\_eq (b1 b2 : bus) : Prop :=

b\_cores b1 = b\_cores b2  
^ (forall j, b\_w b1 j = b\_w b2 j)  
^ (forall j, b\_r b1 j = b\_r b2 j)  
^ (forall j, b\_t b1 j = b\_t b2 j).

**Theorem 8.12** (Independent grants are interchangeable).

*Theorem* independent\_grants\_are\_interchangeable :

forall b i (w : writer) j (r : reader),  
bus\_wf (bus\_r (bus\_w b i w) j r) ->  
bus\_wf (bus\_w (bus\_r b j r) i w).

*Theorem* two\_writes\_to\_one\_cell\_need\_not\_agree :  
~ obs\_eq (bus\_w bus0 0 (mk\_writer true ring\_capacity nil 0))  
(bus\_w bus0 0 empty\_writer).

**Remark 8.13.** The first statement is the multicore law behind “no lock needed”: two writes to *distinct* cells may be performed in either order and the resulting bus is the same. The second is the honesty condition, showing the equivalence is not trivially total — writes to one cell genuinely do distinguish states. Together they are a partial-order reduction: the non-commuting residue is exactly what I9’s induction has to handle, and everything else is quotiented away before any search.

## I13 — The modal reading: laws with no chosen schedule

**Definition 8.12** (Box, diamond, reachability, always).

**Definition** next\_adm (b b' : bus) : Prop :=

exists st, wf\_step st /\ apply\_step b st = b'.

**Definition** necessary (P : bus -> Prop) (b : bus) : Prop :=  
forall b', next\_adm b b' -> P b'.

**Definition** possible (P : bus -> Prop) (b : bus) : Prop :=  
exists b', next\_adm b b' /\ P b'.

**Inductive** reachable (from : bus) : bus -> Prop :=  
| R\_here : reachable from from  
| R\_next : forall b b', reachable from b -> next\_adm b b' ->  
reachable from b'.

**Definition** always (P : bus -> Prop) (b : bus) : Prop :=  
forall b', reachable b b' -> P b'.

**Theorem 8.13** (*bus\_wf* holds always, by the rule that proves it). *Theorem* always\_induction : forall P b,

P b -> (forall x, P x -> necessary P x) -> always P b.

*Theorem* bus\_wf\_is\_always : forall b, bus\_wf b -> always bus\_wf b.

*Corollary* always\_within\_the\_pool : forall b, bus\_wf b ->  
always (fun x => total\_held x <= sector\_pool) b.

**Remark 8.14.** This is the section that makes the paper’s fourth claim precise: the invariant survives *whatever* the hardware picks next, not a schedule the prover chose. The skeleton is proved for its own sake — K, distribution over  $\wedge$  and  $\vee$ , box–diamond duality,  $\top$ , 4 — and one group of witnesses shows *wf\_step* is load-bearing by making the box over the unrestricted relation false (§6.4).

## I14 — The schedule is genuinely undetermined

**Theorem 8.14** (Three successors, no function, observable order).

*Theorem* pub\_bus\_branches\_three\_ways : exists s1 s2 s3 : bus,  
next\_adm pub\_bus s1 /\ next\_adm pub\_bus s2 /\ next\_adm pub\_bus s3  
^ ~ obs\_eq s1 s2 /\ ~ obs\_eq s1 s3 /\ ~ obs\_eq s2 s3.

*Theorem* the\_alphabet\_is\_not\_a\_function :

~ (forall b b1 b2, next\_adm b b1 -> next\_adm b b2 -> b1 = b2).

*Theorem* the\_order\_is\_observable : ~ obs\_eq sub\_then\_snd  
snd\_then\_sub.

**Remark 8.15.** Negative statements, and the most useful ones in the file: they are the non-vacuity obligations for I12–I13. Without them a reader could object that the “modal” layer is an interleaving simulator wearing a hat, or that  $\square$  is true because only one step exists. Here the branching the hardware has, the model has — three pairwise distinguishable successors from one state, a step relation that is not a function, and an order of protocol calls that a bus observation can see — and the invariants of I9 and I13 still hold.

## I15 — The visibility gap: store, then publish

**Definition 8.13** (The two phases of one producer step).

**Definition** evt\_store (e : evtq) (v : nat) : evtq :=  
evt\_write e (evt\_idx (q\_tail e)) (Some v).

**Definition** evt\_publish (e : evtq) : evtq :=  
mk\_evtq (q\_buf e) (q\_head e) (S (q\_tail e)).

**Theorem 8.15** (A floor, not a window; and the phases cannot be swapped). *Theorem* the\_watermark\_is\_a\_floor :

forall b, always (fun x => forall rid,  
r\_live (b\_r b rid) = true ->  
r\_live (b\_r x rid) = true /\ r\_cur (b\_r b rid) <= r\_cur (b\_r x  
rid)) b.

*Theorem* an\_unpublished\_store\_is\_invisible : forall e v,  
q\_head e = q\_tail e -> pr2 (evt\_get (evt\_store e v)) = None.

*Theorem* publishing\_delivers\_what\_was\_stored : forall e v,  
q\_head e = q\_tail e -> pr2 (evt\_get (evt\_publish (evt\_store e v))  
) = Some v.

*Theorem* the\_phases\_cannot\_be\_swapped :  
pr2 (evt\_get (evt\_store (evt\_publish evtq0) 7)) = None.

**Remark 8.16.** A published queue fails in two opposite ways: consuming bytes that were reserved but never committed, and committing without ever publishing. This group separates the two and rules out each, which is exactly the release ordering the target’s ARM cores require and the ordering the C implementation must reproduce with a

barrier. The first statement adds the dual guarantee for the subscriber: its cursor is a monotone floor, so a slow reader loses items but never gains a phantom one, and the modality is what makes that a claim about every future state.

## I16 — Bounded catch-up: a drain budget with an exact effect

**Definition 8.14** (Outstanding items, and a drain of  $k$  reads). *Definition* pending  $b$  ( $r$  : reader) : nat := w\_enq (b\_w b (r\_wid r)) - r\_cur r.

*Fixpoint* read\_k ( $b$  : bus) ( $rid$  : nat) ( $k$  : nat) : bus :=  
 match  $k$  with  
 | 0 => b  
 | S j => read\_k (fst (rcv b NoTask rid)) rid j  
 end.

**Theorem 8.16** (Exactly  $k$  of the gap, and no more than there is). *Theorem* i16\_bounded\_catch\_up : forall  $b$  rid  $k$ ,

rcv\_admit b NoTask rid = true ->  
 pending (read\_k b rid k) (b\_r (read\_k b rid k) rid)  
 = Nat.max 0 (pending b (b\_r b rid) - k).

*Theorem* the\_drained\_reader\_reads\_empty : forall  $b$  rid,  
 rcv\_admit b NoTask rid = true ->  
 pr2 (rcv (read\_k b rid (pending b (b\_r b rid))) NoTask rid) =  
 R\_EMPTY.

*Remark 8.17.* A drain budget now has a worst case one can state to a systems engineer:  $k$  reads close exactly  $k$  of the gap, a budget larger than the gap is inert rather than wrong, and a drained reader’s next call returns R\_EMPTY instead of consuming somebody else’s slot. The supporting obligations are the ones a reviewer should ask for first — read\_k\_neverloses\_the\_cursor (a drain never retires a reader), read\_k\_neverwidens\_the\_gap, and a\_sendwidens\_the\_gap\_of\_its\_own\_subscribers, which says a snd moves exactly its own subscribers’ gaps and no other.

## 8.3 The RTP Theory

Ten groups against oracle invariants R1–R10, 161 obligations, of which 31 are a support layer of list and fold facts proved once so that each contract below is a few lines.

**Definition 8.15** (NuStream static buffer pool).

*Record* buf := mk\_buf { b\_id : nat; b\_rc : nat }.  
*Record* pool := mk\_pool { p\_cap : nat; free\_ids : list nat  
 ; active : list buf; refused : nat;  
 illegal : nat }.  
*Definition* acquire ( $p$  : pool) : pool :=  
 match free\_ids  $p$  with  
 | [] => mk\_pool (p\_cap p) [] (active p) (S (refused p)) (illegal p)  
 | i :: rest => mk\_pool (p\_cap p) rest (mk\_buf i 1 :: active p) (refused p) (illegal p)  
 end.

**Theorem 8.17** (No minting, no double free, exact

recycle). *Lemma* acquire\_never\_mints : forall  $p$  i,  
 In i (active\_ids (acquire p)) -> In i (free\_ids p) ∨ In i (active\_ids p).

*Theorem* unref\_last\_release\_recycles : forall  $p$  i b, pool\_wf p ->  
 find\_buf i p = Some b -> b\_rc b = 1 ->  
 free\_ids (unref i p) = i :: free\_ids p ∧  
 ~ In i (active\_ids (unref i p)) ∧ pool\_conserved (unref i p).

*Remark 8.18.* Use-after-free and double-free, proved at buffer granularity rather than at word granularity: an acquire cannot invent an id outside the fixed set, a shared buffer survives one unref untouched, the release that reaches zero recycles exactly once, and a release of an id not in the pool increments illegal instead of being silently accepted. The conservativity conjunct is the leak claim — release everything and the pool is pool0 again.

**Definition 8.16** (Leaky temporal queue).

*Definition* enqueue ( $t$  : nat) ( $q$  : qstate) : qstate :=  
 if Nat.ltb (length (q\_items q)) queue\_cap then  
 mk\_q (q\_items q ++ [t]) (q\_dropped q)  
 else match q\_items  $q$  with  
 | [] => mk\_q [t] (q\_dropped q)  
 | \_ :: rest => mk\_q (rest ++ [t]) (S (q\_dropped q))  
 end.

**Theorem 8.18** (The window is a hard bound (R2)).

*Lemma* enqueue\_respects\_cap : forall  $t$  q,  
 length (q\_items q) <= queue\_cap -> length (q\_items (enqueue t q))  
 <= queue\_cap.

*Remark 8.19.* Bounded memory under overload, with the overload visible: the jettisoned stamp increments q\_dropped, so a congested path shows up as a counter the monitor can print rather than as an unexplained stall.

**Definition 8.17** (Stamping and dilation).

*Definition* monotone ( $l$  : list nat) : bool :=  
 match  $l$  with [] => true | x :: r => mono\_from x r end.  
*Definition* dilate ( $k$  t : nat) : nat := k \* t.

**Theorem 8.19** (Order survives rescaling (R3, R4)).

*Theorem* dilate\_preserves\_monotone : forall  $k$  l,  
 monotone  $l$  = true -> monotone (map (dilate k)  $l$ ) = true.

*Remark 8.20.* Timestamp monotonicity is the synchronisation claim of the whole media plane, and the group’s companion proves the enforcement half: a regressed or duplicated stamp is refused and leaves the queue identical apart from its refusal counter. Dilation stops being a tuned gain and becomes a proven policy — declaring the capacity rate removes it identically.

**Theorem 8.20** (No interpolated frame claims a future source (R5)). *Definition* outputs\_upto ( $P$  last : nat) :

list nat := map (Nat.mul P) (seq 0 (S (last / P))).

*Theorem* outputs\_never\_ahead : forall  $P$  last  $s$ , In  $s$  (outputs\_upto P last) ->  $s$  <= last.

*Remark 8.21.* The frame-rate-conversion class, including the phantom frame that would read a buffer whose packet has not arrived. outputs\_upto is the emitted cadence as a list, so “exact cadence” and “never ahead” are two readings of the same term.

**Definition 8.18** (Per-plane CPU budget).

*Definition* run\_periods ( $n$  : nat) (use\_tail : bool) (overrun : nat) : plane\_state :=  
 run\_aux n true (if use\_tail then supply\_tail else supply\_whole)  
 overrun plane0.

**Theorem 8.21** (A silent run carries no debt (R6)).

*Theorem* no\_overrun\_leaves\_no\_debt : forall  $n$  (ut : bool),  
 run\_periods  $n$  ut 0 = plane0.

*Remark 8.22.* The admission test for a plane budget, and the proof’s most useful by-product: the shipped INPUT/UI figures balance to the microsecond, so the only headroom in the plan is the sub-millisecond tail, and an overrun without that tail never drains. That is a statement that one specific row of the shipped table is unbuildable by design — found by the proof, not by the review.

**Definition 8.19** (Peer lifecycle on the compositor grid).

```
Definition wf_session (s : session) : Prop :=
length (ss_slots s) = grid_slots /\ count_true (ss_slots s) =
ss_peers s
/\ ss_res s = res_per_peer * ss_peers s.
```

**Theorem 8.22** (Sixteen fill it; a seventeenth is refused (R7)). *Lemma* `join_preserves_wf` : `forall s, wf_session s -> wf_session (fst (join s))`.

*Lemma* `leave_released_preserves_wf` : `forall s i b, wf_session s -> ss_peers s = S b -> nth_error (ss_slots s) i = Some true -> wf_session (fst (leave s i))`.

*Remark 8.23.* Session-table exhaustion is an error code, not corruption, and a departed peer cannot keep a resource slot: the join/leave cycle returns the session to its initial state. `wf_session` ties slots to peers to resources, so a leak in any one of the three is a contradiction rather than an accounting drift.

**Theorem 8.23** (The documented grid mapping is wrong (R8)). *Example* `documented_mapping_cannot_address_the_grid` :

```
cells_on_canvas cell_doc grid_slots <> grid_slots.
Example quarter_grid_tiles_the_canvas_exactly :
grid_slots * (canvas_w / 4) * (canvas_h / 4) = canvas_w *
canvas_h.
```

*Remark 8.24.* The single most concrete argument in the library for writing geometry down before implementing it: the mapping in the shipped document addresses four of the sixteen slots, and the quarter-cell lattice that tiles the canvas exactly was found by disproving the documented one. A proof that fails the specification is worth more than a proof that passes it.

**Theorem 8.24** (The fork is additive and the gate is orthogonal (R9)). *Theorem* `strip_fork_reproduces_upstream` :

```
forall shim, strip (make_fork shim) = pristine.
Theorem gate_orthogonality : forall mp mp' m, media_table mp m =
media_table mp' m.
```

*Remark 8.25.* Fork hygiene as mathematics: stripping the fork recovers the pristine upstream text byte for byte, so “additive only” is checkable mechanically instead of being a review habit, and the media symbol table is constant along the BTRON\_MP axis — the gate adds no symbol and removes none.

**Definition 8.20** (RTP serial arithmetic, grounded in upstream). *Definition* `diff16` (`s1 s2 : nat`) : `nat` :=

```
if Nat.leb s1 s2 then s2 - s1 else seq_mod - s1 + s2.
```

**Theorem 8.25** (The wrap span, and ordered watermarks (R10)).

```
Lemma diff16_forward_is_the_span : forall s1 s2,
s1 <= s2 -> s2 < seq_mod -> diff16 s1 s2 = s2 - s1.
Lemma jb_watermarks_are_ordered : forall probs, jb_low_ms probs <=
jb_high_ms probs.
```

*Remark 8.26.* Twenty-three obligations whose content is that the model’s constants are a deployed implementation’s constants — the 16-bit serial ring and its wrap span, the jitter-buffer full test, the 15 % and 90 % watermarks, the jitter EWMA bound, probation = 2, the `CLOCK_TIME_NONE` sentinel, DTS-over-PTS, the power-of-two mask, video-rate’s forward gate — each cited to an upstream source line. The off-by-one in a wrap or a mask is exactly what no test on a quiet network would surface.

## 8.4 The SMP Substrate

Eight groups, 69 obligations, the proved companion of `media_smp_model.ml`, which runs 52 checks over 279,936 interleavings of the same three cursors.

**Definition 8.21** (One producer lane of a multicursor

```
ring). Record lane_state := mk_lane { l_reserved : nat;
l_published : nat; l_read : nat }.
```

*Definition* `wf_lane` (`s : lane_state`) : `Prop` :=

```
l_read s <= l_published s /\ l_published s <= l_reserved s /\
l_reserved s <= l_read s + capacity /\ l_reserved s <=
l_published s + inflight_max.
```

*Definition* `claim` (`n : nat`) (`s : lane_state`) : `lane_state` :=

```
if claim_admissible n s then { l_reserved := l_reserved s + n
} else s.
```

**Theorem 8.26** (Admission is safe and refusal inert (1)).

```
Lemma claim_preserves_wf : forall n s, wf_lane s -> wf_lane (
claim n s).
```

*Lemma* `claim_refused_is_inert` : `forall n s, claim_admissible n s = false -> claim n s = s`.

*Remark 8.27.* A lost CAS race leaves no residue: the refused claim is the identity on the lane, so a retry loop needs no compensation code. The admitted claim can neither oversubscribe the ring nor outrun its own publish watermark, which is the whole correctness argument for the lock-free claim/publish discipline of decision D1.

**Theorem 8.27** (No tearing (2)). *Lemma*

```
drain_never_reaches_unpublished : forall n s, wf_lane s ->
l_read s + drained n s <= l_published s.
```

*Lemma* `claim_publish_drain_round_trip` : `forall n s, wf_lane s -> claim_admissible n s = true -> drained n (publish (claim n s)) = n`.

*Remark 8.28.* A reader can never consume reserved-but-unpublished bytes, and an accepted claim/publish/drain round trip returns exactly what was claimed — the half-packet class, stated as an invariant of the model rather than as a rule about memory barriers.

**Theorem 8.28** (Three states, none of them blocking

```
(3)). Theorem rcv_is_never_blocking : forall n s, rcv n s <>
RCV_BLOCKED.
```

*Remark 8.29.* Discharged by three case analyses and a `discriminate`. The group also proves each of the three states reachable, which is what keeps the statement from being vacuous: `rcv` can never become an implicit scheduler yield, and it can never be starved by a definition that makes the ready state empty.

**Theorem 8.29** (Byte conservation (4)).

*Lemma* *conservation* : *forall* *s*, *wf\_lane s* ->  
*in\_flight s* = (*l\_published s* - *l\_read s*) + *outstanding s*.  
*Lemma* *drain\_splits\_the\_readable\_pool* : *forall* *n s*,  
*drained n s* + (*l\_published s* - *l\_read s* - *drained n s*) =  
*l\_published s* - *l\_read s*.

*Remark 8.30.* In flight is readable plus still-in-commit, and a drain takes only from the readable pool. This is the equation a monitor prints, which is the point: the counters on the console are the counters that were proved.

**Definition 8.22** (AMP drain ownership).

*Inductive* *core* := *CORE\_BOOT* | *CORE\_IO*.  
*Definition* *owner* (*p* : *amp\_phase*) : *option core* :=  
*match* *p* *with*  
| *PH\_BOOT* | *PH\_RELEASED* | *PH\_TAKEOVER* => *Some CORE\_BOOT*  
| *PH\_IO\_ACTIVE* | *PH\_IO\_STALE* => *Some CORE\_IO*  
| *PH\_DEAD* => *None*  
*end*.  
*Definition* *io\_may\_drain* (*p* : *amp\_phase*) (*released* : *bool*) : *bool*  
:=  
*andb* (*io\_phase p*) *released*.  
*Definition* *boot\_may\_drain* (*p* : *amp\_phase*) (*age heart* : *nat*) : *bool*  
:=  
*andb* (*boot\_phase p*) (*take\_over\_due age heart*).

**Theorem 8.30** (One owner, and no early failover (5)).

*Lemma* *owner\_is\_single\_valued* : *forall* *p c1 c2*,  
*owner p* = *Some c1* -> *owner p* = *Some c2* -> *c1* = *c2*.  
*Lemma* *io\_cannot\_drain\_before\_release* : *forall* *p*, *io\_may\_drain p*  
*false* = *false*.  
*Lemma* *boot\_and\_io\_cannot\_both\_drain* : *forall* *p released age heart*,  
*io\_may\_drain p released* = *true* -> *boot\_may\_drain p age heart* =  
*false*.  
*Lemma* *takeover\_not\_premature* : *forall* *age heart*,  
*stale age heart* < *timeout\_ms* -> *take\_over\_due age heart* = *false*.

*Remark 8.31.* The shared accumulator between planes has one owner by construction, the I/O core cannot touch it before the release word, and failover cannot fire before the timeout bar. The last of the four is the one that prevents the classical dual-consumer corruption: a takeover that reads a timestamp too optimistically and drains while the other core is still draining.

**Definition 8.23** (Seqlock slot).

*Record* *slot* := *mk\_slot* { *sl\_seq* : *nat*; *sl\_acc* : *nat* }.  
*Definition* *stable* (*s* : *slot*) : *Prop* := *exists* *k*, *sl\_seq s* = *2 \* k*.  
*Definition* *consumer\_delta* (*cur before* : *slot*) : *nat* := *sl\_acc cur*  
- *sl\_acc before*.

**Theorem 8.31** (Lossless coalescing (6)).

*Lemma* *publish\_is\_monotonic* : *forall* *d s*, *sl\_acc s* <= *sl\_acc* (*publish\_delta d s*).  
*Lemma* *delta\_nonneg* : *forall* *cur before*, *sl\_acc before* <= *sl\_acc cur*  
->  
0 <= *consumer\_delta cur before*.  
*Lemma* *guarded\_rejects\_unstable* : *forall* *a b*, *seq\_matches a b* = *true*  
->  
*updating a* = *true* -> *guarded a b* = *None*.

*Remark 8.32.* Truncated subtraction is the hazard here: a lagging consumer’s delta must never be negative, and monotonicity is what makes *sub* faithful. A torn read is not corrected — it is rejected, returning no delta at all, which is the only safe answer an interrupt-unfriendly core can give.

**Theorem 8.32** (Refinement when the gate is off (7)).

*Theorem* *mp\_gate\_is\_additive* : *forall* *b ring\_ok s d*, *mp\_step b*  
*ring\_ok s d* = *mp\_off\_step s d*.  
*Theorem* *gate\_does\_not\_change\_observed\_delta* : *forall* *b ring\_ok s d*,  
*fst* (*mp\_step b ring\_ok s d*) = *fst* (*legacy\_step s d*) /\  
*snd* (*mp\_step b ring\_ok s d*) = *snd* (*legacy\_step s d*).

*Remark 8.33.* With BTRON\_MP=0 the asynchronous path is the legacy path, step for step, and the ring is not even an input to it. Shipping a flag off is normally a promise about a build; here it is discharged by refinement, which removes the regression risk that made the flag politically difficult in the first place.

**Theorem 8.33** (The shipped constants instantiate the contracts (8)).

*Example* *one\_byte\_claim\_is\_admitted\_at\_shipped\_geometry* :  
@*claim\_admissible* 4 2 1 *catch\_up\_lane* = *true*.  
*Lemma* *a\_lane\_at\_capacity\_refuses\_any\_positive\_claim* : *forall* *n s*,  
*l\_reserved s* = *l\_read s* + 4 -> 0 < *n* -> @*claim\_admissible* 4 2 *n s*  
= *false*.  
*Example* *shipped\_ladder\_warns\_first* : *forall* *age heart*,  
@*take\_over\_due* 100 *age heart* = *true* -> @*warn\_due* 50 *age heart* =  
*true*.

*Remark 8.34.* Ring 4/2 and the 50/100 ms heartbeat bars, written as closed obligations rather than as numbers in a comment — and the group names the inequality the proofs actually need, so a future retune knows which bound it is allowed to move.

## 8.5 Hypermedia, Direct Manipulation, and TAD

61 obligations over the Real/Virtual Body model, the drag-and-drop protocol, and the persistence format. The two headline results are the integrity of a drop and the bijectivity of the encoding.

**Definition 8.24** (Registry, integrity, and the sentinel).

*Definition* *storage* := *list robj*.  
*Definition* *body\_registered* (*s* : *storage*) (*i* : *robj\_id*) : *Prop* :=  
*exists* *r*, *In r s* /\ *r\_id r* = *i*.  
*Definition* *registry\_wf* (*s* : *storage*) : *Prop* := *forall* *r*, *In r s* ->  
*r\_id r* < 0.  
*Definition* *doc\_integrity* (*s* : *storage*) (*doc* : *clarity\_doc*) : *Prop*  
:=  
*forall* *f*, *In f* (*d\_frames doc*) -> *forall* *v*, *In v* (*f\_vobjs f*) ->  
*body\_registered s* (*v\_target v*).

*Remark 8.35.* Id 0 is the TAD “no Real Body” sentinel, so the first conjunct is what makes the on-disk encoding unambiguous, and *doc\_integrity* is the serializer’s precondition stated as a property of a document rather than as an assertion in a writer.

**Theorem 8.34** (A drop cannot dangle or lose a frame).

*Theorem* *and\_drop\_into\_preserves\_integrity* : *forall* *s doc fid*  
*r*,  
*In r s* -> *doc\_integrity s doc* -> *doc\_integrity s* (*drop\_into doc*  
*fid r*).

*Remark 8.36.* The hypermedia defect class “an icon that opens nothing”: a Virtual Body can only ever be bound to a registered Real Body, because the operation that performs the binding is proved to preserve the property that makes the binding sound.

**Definition 8.25** (The protocol’s state machine).

```

Inductive dnd_state :=
| DND_Idle
| DND_Dragging (r : robj) (pt : point)
| DND_Hovering (r : robj) (target_fid : robj_id) (pt : point).
Definition dnd_drop (st : dnd_state) (doc : clarity_doc) :
  clarity_doc * dnd_state :=
  match st with
  | DND_Idle      => (doc, DND_Idle)
  | DND_Dragging r pt => (canvas_drop r pt doc, DND_Idle)
  | DND_Hovering r fid _ => (drop_into doc fid r, DND_Idle)
end.

```

**Theorem 8.35** (The GUI cannot lock up (8)).

```

Theorem dnd_drop_returns_idle : forall st doc, snd (dnd_drop st doc) = DND_Idle.
Theorem dnd_no_stranded_hover : forall st doc r fid pt,
  snd (dnd_drop st doc) <> DND_Hovering r fid pt.

```

*Remark 8.37.* Every drop and every cancel lands in `DND_Idle`, and no drop can leave a hover on a frame that has moved. Stranded state — a cursor that thinks it is still dragging, a session that can only be escaped by restarting — is the class of defect users report and testers never reproduce.

**Theorem 8.36** (No type confusion at the persistence

```

boundary). Definition r_is_draw (r : robj) : bool :=
  match r.kind r with VObjDraw => true | _ => false end.
Theorem drop_onto_never_rebinds_to_a_non_draw_body : forall fid r f
  ,
  r_is_draw r = false -> f_img (drop_onto fid r f) = f_img f.

```

*Remark 8.38.* An Image Frame’s bound Real Body stays a registered Draw Body, and a body of another kind leaves the binding untouched: a sound or text body can never be re-read as an image, which is the class of bug that makes a saved document unopenable.

**Definition 8.26** (TAD streams).

```

Definition encode_stream (doc : clarity_doc) : tad_stream :=
  mk_tad (d_fmt doc) (map encode_frame (d_frames doc)).
Definition decode_stream (t : tad_stream) : clarity_doc :=
  mk_doc (ts_fmt t) (map decode_frame (ts_frames t)).
Definition doc_norm (doc : clarity_doc) : Prop :=
  forall f, In f (d_frames doc) -> frame_norm f.

```

**Theorem 8.37** (The format is a bijection (9)).

```

Theorem tad_roundtrip_isomorphic : forall doc, doc_norm doc ->
  decode_stream (encode_stream doc) = doc.
Theorem tad_encoding_is_injective : forall d1 d2, doc_norm d1 ->
  doc_norm d2 ->
  encode_stream d1 = encode_stream d2 -> d1 = d2.

```

*Remark 8.39.* Round trip and injectivity together are the interchange claim the cleanroom exists for: no document is lost or invented, and any conforming reader is adequate. The final group of the file (13 obligations) proves *doc\_norm* for the concrete documents the oracle manipulates, so the theorems are about the states that actually run rather than about a subclass test data happens to fall into.

## 8.6 The B-FS Theories

The smallest and oldest part of the library, and the most concrete: a journal with a write-ahead log, a parameterised B+Tree, and a 64-bit block allocator.

**Definition 8.27** (Volume, journal, crash, recovery).

```

Record journal := mkJournal { log : list mutation; committed
  : list mutation; in_tx : bool }.
Record volume  := mkVolume { gen : generation; dirty : bool; j :
  journal }.
Definition well_formed (v : volume) : Prop := (j v).(in_tx) =
  false -> (j v).(log) = [].
Definition crash (v : volume) : volume := { | gen := v.(gen);
  dirty := true
  ; j := { | log := []; committed := (j v).(committed); in_tx :=
  false | } }.
Definition replay (v : volume) : volume := { | gen := v.(gen);
  dirty := false
  ; j := { | log := []; committed := (j v).(committed); in_tx :=
  false | } }.

```

**Theorem 8.38** (Crash consistency, and recovery that is idempotent).

```

Theorem crash_replay_safety : forall v,
  is_clean (replay (crash v)) /\
  (j (replay (crash v))).committed = (j v).committed.
Theorem recovery_idempotent : forall v, replay (crash (replay (
  crash v))) = replay (crash v).
Theorem steps_preserve_wf : forall v v', well_formed v -> steps v v'
  -> well_formed v'.

```

*Remark 8.40.* There is no “recovery failed” branch at boot, running recovery twice is running it once, and the state machine cannot be driven into an undefined phase. The illegality of an out-of-sequence call is a returned `option` — four lemmas say `begin_tx` refuses a nested transaction and `log_mut`, `commit` and `abort` refuse outside one — so an error path is inert rather than corrupting.

**Theorem 8.39** (What the WAL exists for).

```

Theorem log_then_crash_not_durable : forall v v_tx v_log m,
  begin_tx v = Some v_tx -> log_mut v_tx m = Some v_log ->
  (j (replay (crash v_log))).committed = (j v).committed /\
  (j (replay (crash v_log))).log = [].
Theorem crash_during_tx_recovery : forall v v_tx, begin_tx v = Some
  v_tx ->
  is_clean (replay (crash v_tx)) /\ gen (replay (crash v_tx)) = gen
  v.
Theorem step_generation_ge : forall v v', step v v' -> gen v' >=
  gen v.

```

*Remark 8.41.* Uncommitted log never enters durable history, a crash mid-transaction restores the pre-transaction state, and the generation never decreases — so the defect “the file looked right until the power cut” and its cousins, ABA reuse of a transaction id and time-travelling generations, are excluded. The group’s two `Example` obligations are the audited scenarios themselves, closed by computation, which is what lets a console audit re-run them.

**Definition 8.28** (Sorted leaf).

```

Inductive sorted : list key -> Prop :=
| sorted_nil : sorted []
| sorted_one : forall x, sorted [x]
| sorted_cons : forall x y l, x < y -> sorted (y :: l) -> sorted
  (x :: y :: l).
Fixpoint lookup (k : key) (l : list (key * fid64)) : option fid64
:=
  match l with [] => None | (x, v) :: xs => if Nat.eqb k x then
    Some v else lookup k xs end.

```

**Theorem 8.40** (A stored key stays findable).

```

Lemma insert_sorted_preserves_sorted : forall l k, sorted l
  -> sorted (insert_sorted k l).
Lemma lookup_insert_same : forall l k v, lookup k (insert_kv k v l)
  = Some v.

```

```

Lemma lookup_insert_other : forall l k1 k2 v, k1 <> k2 ->
  lookup k1 (insert_kv k2 v l) = lookup k1 l.
Theorem split_preserves_elements : forall (A : Type) (n : nat) (l :
  list A),
  let (left, right) := split_at n l in left ++ right = l.

```

*Remark 8.42.* Three silent-data-loss classes in four statements: an insertion that breaks order so a binary probe misses an extent, an update that drops the key it overwrote or disturbs its neighbours, and a node overflow that loses a record on the split. *split\_preserves\_elements* is the non-loss law for the second of these, and *valid\_sibling\_pair* the corresponding statement that a *readdir* walking the sibling chain sees every record exactly once.

**Definition 8.29** (Block runs and the 64-bit FID).

```

Record block_run := mkRun { run_ag : ag_id; run_start :
  block_offset; run_len : run_length }.
Definition blocks_per_ag : nat := 256 * 256.
Definition encode_fid (r : block_run) : nat := r.(run_ag) *
  blocks_per_ag + r.(run_start).
Definition decode_ag (fid : nat) : ag_id := fid / blocks_per_ag.
Definition decode_start (fid : nat) : block_offset := fid mod
  blocks_per_ag.
Definition runs_disjoint (r1 r2 : block_run) : Prop :=
  r1.(run_ag) <> r2.(run_ag) /\
  r1.(run_start) + r1.(run_len) <= r2.(run_start) /\
  r2.(run_start) + r2.(run_len) <= r1.(run_start).

```

**Theorem 8.41** (Faithful encoding; no two runs overlap).

```

Theorem fid_encode_decode_inv : forall r, blocks_per_ag <> 0
->
  r.(run_start) < blocks_per_ag ->
  decode_ag (encode_fid r) = r.(run_ag) /\ decode_start (encode_fid
    r) = r.(run_start).
Theorem no_self_disjoint_positive : forall r, r.(run_len) > 0 -> ~
  (runs_disjoint r r).
Theorem alloc_in_bounds_preserved : forall (runs : list block_run)
  (new_r : block_run),
  (forall r, In r runs -> run_in_bounds r) -> run_in_bounds new_r
->
  forall r, In r (new_r :: runs) -> run_in_bounds r.

```

*Remark 8.43.* Two files never share clusters, a run never overlaps itself, nothing is allocated outside its allocation group, and the cluster-to-FID conversion is invertible on in-range runs — which is the statement that the free-space figure in the volume header can be trusted without *fsck*. *total\_blocks* adds the third: the used-block count is conserved by extending the run list.

## 8.7 What the Ledger Is For

Three uses, and only the first is a number.

(i) *Defect classes, not test coverage.* Every remark above names a failure a reviewer would otherwise have to be reassured about: late joiners pinning history, a torn seqlock read, two files on one cluster, an icon that opens nothing, a budget that never recovers. The grouping is chosen so a claim can be challenged item by item — and the two entries whose proofs found something the document had wrong (R8’s four-of-sixteen cells, R6’s unbuildable tail) are the argument that this was not ceremony.

(ii) *An audit artefact.* The 39 `Print Assumptions` lines are the headline obligations a reader may ask the kernel

about directly, and `coqchk -o` then re-validates the compiled module and must answer `<none>` four times (§7.3). That is machine-checkable by someone who does not read COQ — which is the difference between a proof and a proof modulo wishes.

(iii) *A regression budget.* Once each entry’s statement is published, weakening it is visible: deleting a hard case changes the obligation count, and moving a hypothesis into an *Axiom* makes `coqchk` speak. Two omissions are recorded here rather than hidden. Three files still lack the in-file gate and are audited by `coqchk` alone. And the harness array also registers `media_properties.v` — the global model in which the ATR/VP object semantics and the OS task states share the bus with the message layer — which is authored nowhere, is written as continuing work (§9), and is reported by the harness as `FAIL`, never as a quiet pass.

## 9 Conclusion

The four tiers of the BTRON concurrency model, the media stack above it, and the InterCore protocol between its cores are one algebra: a total state transformer over a table-shaped bus with derived accounting, a watermark fold as the shared retention operator, boolean admission read as an internal modality, and commutation as the reduction that makes non-determined interleavings tractable. What the exercise in CIC contributed is not the theorems—each was expected—but three design decisions that only a checker insists upon:

1. **Broker-freeness is a definability result.** The multi-consumer view had to be written as a *projection* (Equation (5)) for the conservation and provenance lemmas to close without shared mutable state. A broker would have been representable; it would not have been provable at the same price.
2. **Non-blocking is a type-level commitment.** Once `rcv` had to return a value in all three cases, “never yields” became a case analysis, and the priority-inversion argument became Theorem 5.1 rather than a paragraph about scheduler policy.
3. **Total accounting removes a bug class.** Deriving *enq*, *deq*, *held* and *pool-free* as lengths and minima (Equation (2)) made state-reachability disagreement between counters unnameable—and, twice, made the model correct where the C prototype was not.

Work continuing along this line has a name in the tree already. The harness registers eight oracles and eight theory files; one of the latter, `media_properties.v`, is the file the corpus does not yet contain, and the `FAIL` that prints for it is the line the ledger of Section 8 records. That file is the *global* model: one bus on which the Real/Virtual Body semantics, the OS task states, and the message layer are

coordinates of the same state transformer, so that the four claims of this paper are proved together rather than four times. Its companion is the transfer of the shipped-value arithmetic from oracle to theory via binary numerals with a proven bridge, which would let  $10^6$ -scale launch figures be asserted where they are today only executed.

(BTRON 3.20),” Synrc Research Report, companion to this paper, 2026.

- [13] M. Sokhatsky, “B-System Virtualization and Formal Foundations: A Cleanroom Architecture for Real-Time Operating Systems from Embedded MCUs to Modern Hypervisors,” Synrc Research Report, 2026.

## References

- [1] K. Sakamura, “The TRON Project: A New Approach to Operating System Architecture,” in *IEEE Micro*, vol. 7, no. 2, pp. 8–14, 1987.
- [2] BTRON3 Working Group, “BTRON3 Specification Ver 3.20.00,” TRON Association, Tokyo, Japan, 1994.
- [3] B. Barras, S. Boutin, C. Cornes, J. Courant, J.-C. Filliâtre, E. Giménez, H. Herbelin, G. Huet, C. Muñoz, C. Paulin-Mohring, A. Saïbi, B. Werner, “The Coq Proof Assistant Reference Manual,” INRIA Research Report RT-0203, 1997.
- [4] The Coq Development Team, “The Coq Proof Assistant Reference Manual, version 9.2,” INRIA, <https://coq.inria.fr/doc>, 2025.
- [5] X. Leroy, D. Doligez, A. Frisch, J. Garrigue, D. Rémy, J. Vouillon, “The OCaml System, Release 5.5: Documentation and User’s Manual,” INRIA, 2024.
- [6] L. Lamport, “Proving the Correctness of Multiprocessors,” in *IEEE Transactions on Software Engineering*, vol. SE-3, no. 2, pp. 125–128, 1977.
- [7] V. Diekert and G. Rozenberg (eds.), *Traces: Semantics for Concurrency*, World Scientific, Singapore, 1997.
- [8] J. Armstrong, “Making reliable distributed systems in the presence of software errors,” Ph.D. dissertation, Royal Institute of Technology, Stockholm, Sweden, 2003.
- [9] G. Klein, K. Elphinstone, G. Heiser, et al., “seL4: Formal Verification of an OS Kernel,” in *Proceedings of the 22nd ACM Symposium on Operating Systems Principles (SOSP)*, pp. 207–220, 2009.
- [10] H. Schulzrinne, S. Casner, R. Frederick, M. Jacobson, “RTP: A Transport Protocol for Real-Time Applications,” RFC 3550, IETF, 1994.
- [11] GStreamer Project, “GStreamer Core Library and Base Plugins: pads, capabilities, segments, and buffer reference counting,” <https://gstreamer.freedesktop.org/documentation/>, accessed 2026.
- [12] M. Sokhatsky, “Microkernels, Unikernels, and the Actor Core: Architectural Trade-Offs, Real-Time Determinism, and SMP Subsystem Isolation in B-System